Lab 2: Music Genre Classification with Convolutional Neural Networks

Anna Casanovas Poirier
Machine Learning for Sound and Music

November 5, 2026

Contents

1 Introduction **3**

2 Data and Experimental Setup **4**
2.1 Dataset and Spectrogram Preprocessing 4
2.2 Train/Validation/Test Partition 4
2.3 Evaluation Protocol and Statistical Tests 5

3 Task 1: CNN Trained from Scratch **6**
3.1 Architecture and Training 6
3.2 Configuration Search 6
3.3 Final Model and Test Results 7
3.4 Error Analysis of the CNN 8

4 Task 2: Fine-Tuning Pretrained Models **11**
4.1 Vision Models (VGG16, ResNet50, EfficientNetB0) 11
4.2 Audio Model (VGGish) 11
4.3 Fine-Tuning Strategies 12
4.4 Results 12

5 Lab 1 Models Retrained on the Shared Partition **16**
5.1 Descriptors and Lab 1 Pipeline 16
5.2 Retraining Protocol 16
5.3 Model Selection by Cross-Validation 16
5.4 Results 17
5.5 Five Partitions 18

6 Comparison of All Models **19**
6.1 Five Partitions 19
6.2 Main Partition: Every Model on the Same 200 Songs 20
6.3 Statistical Comparison of the Models Trained on Five Partitions 21
6.4 How the Three Approaches Fail 21

7 Research Questions **23**
7.1 Mandatory Questions 23
7.1.1 Q1. Do learned features beat hand-crafted descriptors? 23
7.1.2 Q2. Do CNNs and descriptor models make the same mistakes? 24
7.1.3 Q5. How much does one 80/20 split tell you? 26

7.2 Additional Questions 28
7.2.1 Q3. Are the two approaches complementary? 28
7.2.2 Q4. Is the extra cost worth it? 30
7.2.3 Q6. How does data size affect the comparison? 31
7.2.4 Q9. Would fine-tuning a pretrained model improve the model? 32
7.2.5 Q10. Frozen embeddings + traditional ML algorithm, or full fine-tuning? 33

8 Conclusions **35**

A Complete Results of All Trained Models **37**
A.1 Lab 1 pipelines (cross-validation) 37
A.2 All models on the main partition 38
A.3 Pairwise comparisons of the models trained on five partitions 38

B Hyperparameters and Configurations **39**

Code

All the code, the notebooks for training, evaluating and all the results are in the following public github repository: `https://github.com/annacp04/lab2-ML-music-classification`. The README.md clearly explains where everything to check the code or results comented in this Documentation.

1 Introduction

Coming from Lab1 we have already seen what automatic music genre classification in music information retrieval means. From a short segment of audio, a label like *jazz* or *metal* are detected. This can be done with hand-crafted descriptors from the signal (spectral shape, MFCCs, chroma, rhythm statistics), feeding them to a classical classifier (approach of Lab1) or directly using an image representation of the audio. This second way involves using Convolutional Neural Networks or pretrained models and use the Spectrogram Image directly to classify.

These CNN approach is studied in this second lab on the GTZAN dataset and the results are compared with the first lab. Task 1 trains a small CNN from scratch on mel-spectrogram images and optimizes its configuration (depth, number of filters, regularization, kernel shape and input size). Task 2 fine-tunes networks pretrained on ImageNet (VGG16, ResNet50 and EfficientNetB0) with different amounts of unfrozen layers, using the same training and test data. Finally, the Lab 1 models, which use hand-crafted descriptors of the same songs, are retrained on exactly the same partition so that all approaches can be compared song by song.

Beyond reporting accuracies, the lab poses a set of research questions like: Do learned features beat hand-crafted descriptors? Do both kinds of models make the same mistakes? How much does a single 80/20 split tell?

Because the test set contains only 200 songs (one song is 0.5 percentage points), a single split can easily suggest a spurious winner. Every final model is therefore evaluated on five different 80/20 partitions, always with the same songs for all models, and models are compared with paired statistical tests (McNemar and a corrected resampled t-test) rather than by raw accuracy differences. Hyperparameters are chosen using training data only (a validation split or cross-validation), and the test set is used once per final configuration.

2 Data and Experimental Setup

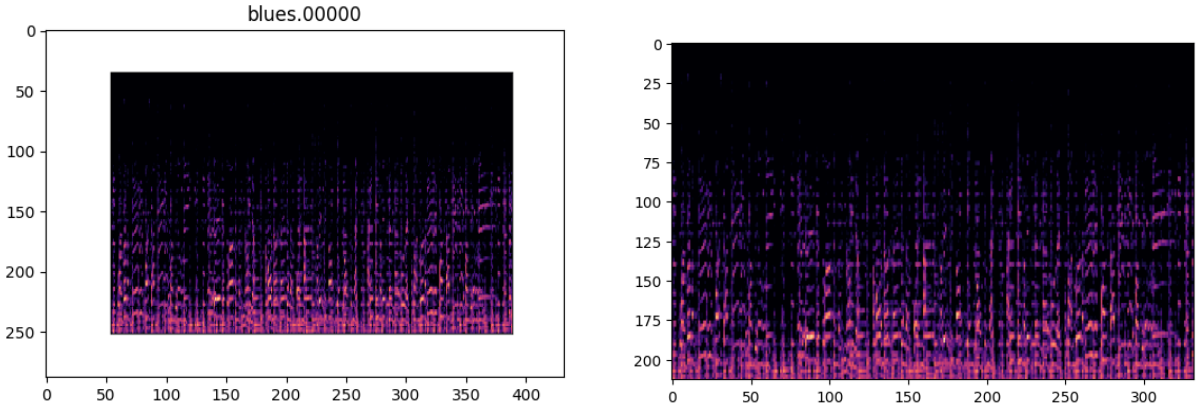
2.1 Dataset and Spectrogram Preprocessing

All experiments use the GTZAN music genre dataset: ten genres (blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae and rock) with 100 audio clips of 30 s each. The spectrogram set provided for this lab contains 999 songs, jazz has 99 clips and every other genre has 100. The main focus of this lab is using Mel-Spectrogram Images as representations of the songs.

The provided images are 432×288 RGBA PNG renderings of a colour-mapped mel spectrogram, drawn inside a white figure frame (Figure 1a). The frame is identical for all songs and carries no information, so the following preprocessing is applied to every image:

1. Crop to the rectangle with corners $(55, 37)$ and $(387, 250)$ pixels, which keeps only the spectrogram (332×213 pixels; Figure 1b).
2. Convert RGBA to RGB.
3. Resize with bilinear interpolation to 128×192 pixels (frequency $\times$ time) for the CNN trained from scratch, and to 160×240 pixels for the pretrained networks (Section 4).
4. Standardise each colour channel with the mean and standard deviation computed on the *training* songs of the corresponding partition only.

The images are 8-bit colour-mapped renderings, not the underlying log-mel matrices, so some dynamic-range information is lost; this should be kept in mind when comparing them with descriptors computed directly from the audio.



(a) Original image (432×288 px).

(b) After cropping (332×213 px).

Figure 1: Spectrogram of `blues.00000` before and after removing the figure frame. Axes are in pixels.

Hand-crafted descriptors (Lab 1). The file `features_30_sec.csv` contains one row per 30-s clip with global descriptors computed from the audio: chroma, RMS energy, spectral centroid, bandwidth and roll-off, zero-crossing rate, harmonic and percussive components, tempo, and the mean and variance of 20 MFCCs. After removing constant columns, 58 descriptors remain. This dataset is only used as a comparison with Lab1

2.2 Train/Validation/Test Partition

The data are split *by song* into 80 % training (799 songs) and 20 % test (200 songs), stratified by genre. Each song appears in exactly one of the two sets; in particular the Lab 1 models were

retrained on the same partition as the Lab 2 models (the Lab 1 preprocessing pipeline accepts an explicit list of test songs), so that all results are comparable song by song. The test set is never used to choose hyperparameters. Model selection uses a validation set (or cross-validation) carved out of the training songs only, Table 1 summarises what each stage uses.

Stage	Selection procedure	Train	Val.	Test
Task 1 (CNN)	single validation split (20 % of the training songs)	639	160	200
Task 2 (fine-tuning)	single validation split (10 % of the training songs)	719	80	200
Lab 1 (descriptors)	10-fold stratified group cross-validation in the training songs	799	–	200

Table 1: Number of songs used at each stage (main partition). Final models are retrained on the 799 training songs when the protocol requires it (Task 1 and Lab 1). The fine-tuned networks keep early stopping on their validation split.

The random seed 42 partition is used for model selection, but as it has been said, to measure how much a single partition can mislead, the final configurations are repeated on five different partitions (seeds 42–46), always with the same songs for every model. Partitions are stored as lists of song identifiers and shared across all notebooks. Hyperparameters were chosen on the main partition and are kept fixed in the other four. They are slightly optimistic for seed 42, and the test songs of the other seeds partly overlap with the songs used for selection. This limitation applies equally to all models.

2.3 Evaluation Protocol and Statistical Tests

The main metric is accuracy on the 200 test songs, macro-averaged F1 and confusion matrices are used as complements. With 200 songs, one song is worth 0.5 percentage points (pp) and the standard error of an accuracy near 0.8 is about 2.8 pp, so Wilson 95 % intervals are reported for single-partition results, and mean ± sample standard deviation ($n - 1$) over the five partitions otherwise. Two models are compared on the same test songs with the exact McNemar test (two-sided binomial test on the songs where exactly one of the two models is correct), with Holm's correction when several pairs are tested. Across the five partitions, differences in accuracy are tested with the corrected resampled t-test of Nadeau and Bengio, which inflates the variance by a factor $1/k + n_{\text{test}}/n_{\text{train}}$ ($k = 5$, $n_{\text{test}}/n_{\text{train}} = 0.25$) to account for overlapping training sets. With only five partitions this test has low power, so a non-significant result is not evidence of equivalence.

About reproducibility, Python, NumPy and TensorFlow seeds are fixed, but GPU kernels are not fully deterministic. The same CNN configuration and seed gave test accuracies of 0.800, 0.805, 0.810 and 0.825 in four separate runs, so differences of 2–3 pp between single runs should not be interpreted.

3 Task 1: CNN Trained from Scratch

3.1 Architecture and Training

The network is a stack of identical convolutional blocks followed by a classifier. Each block is a 3×3 convolution (same padding), batch normalisation, ReLU and 2×2 max pooling. After the last block, global average pooling collapses the time–frequency map into one vector per song, followed by dropout and a dense layer with 10 outputs (logits). Global average pooling keeps the parameter count small (no large dense layer on a flattened map) and makes the classifier independent of the exact input size. The input has shape $128 \times 192 \times 3$ (frequency $\times$ time $\times$ colour channels); the three channels are kept because the PNG images are colour-mapped, and this lets the same data feed the pretrained networks of Task 2. The depth and number of filters are explored in Section 3.2.

The network is trained with the AdamW optimiser (weight decay 10^{-4}), a cosine learning-rate schedule starting at 10^{-3} and spanning 40 epochs, batch size 32 and the cross-entropy loss on the logits. The training images are augmented on the fly with a SpecAugment-style masking: one band of up to 16 consecutive frequency rows and one band of up to 24 consecutive time columns (12.5 % of each axis) are set to the mean value (zero after standardisation), at random positions that change for every song and every epoch. Validation and test images are never augmented. During the configuration search, early stopping on validation accuracy (patience 10, best weights restored) is used; the final model is retrained without a validation set for a fixed number of epochs (Section 3.3).

3.2 Configuration Search

The configuration is chosen using only the validation split of 160 songs (Table 1) in two stages. Stage 1 varies the depth and number of filters (six networks) with the remaining choices fixed (dropout 0.3, augmentation on, learning rate 10^{-3}, 3×3 kernels, 128×192 input). Stage 2 starts from the best stage-1 network (four blocks with 32–256 filters) and changes one factor at a time: dropout, augmentation, learning rate, kernel shape and input size. Table 2 lists the 16 trained configurations.

Stage	Configuration	Parameters	Val. acc.	Best epoch
1	2 blocks (16-32)	5,514	0.463	24
1	3 blocks (16-32-64)	24,458	0.594	38
1	3 blocks (32-64-128)	94,986	0.688	37
1	4 blocks (16-32-64-128)	99,210	0.744	40
1	4 blocks (32-64-128-256) [base]	391,946	0.763	40
1	5 blocks (32-64-128-256-256)	982,538	0.588	19
2	dropout 0.0	391,946	**0.775**	39
2	dropout 0.5	391,946	0.606	21
2	no augmentation	391,946	0.756	37
2	learning rate $3 \cdot 10^{-4}$	391,946	0.719	36
2	learning rate $3 \cdot 10^{-3}$	391,946	0.713	40
2	kernel 5×5	1,081,610	0.738	40
2	kernel 7×3 (freq. $\times$ time)	909,194	0.400	4
2	kernel 3×7 (freq. $\times$ time)	909,194	0.381	7
2	input 64×96	391,946	0.725	38
2	input 192×288	391,946	0.750	38

Table 2: Configuration search on the validation split (160 songs). The selected configuration is in bold. The base for stage 2 is the best stage-1 network.

A few observations follow, bearing in mind that one validation song is worth 0.6 pp and the standard error of these accuracies is about 3.4 pp. First of all, Most stage-2 changes are not distinguishable from the base. Dropout 0.0, no augmentation and the larger input are within ±1.3 pp of the base (at most two songs). The selected configuration (dropout 0.0, 0.775) is therefore a statistical tie with the base (0.763); it is kept because the selection rule is the validation accuracy and the test set is not consulted. Larger deviations appear for the 5×5 kernel (–2.5 pp), the smaller input (–3.8 pp) and both learning rates (–4.4 and –5.0 pp), and even these are only one to one and a half standard errors. From this we can conclude that maybe those parameters affect more in setting were there is more underfitting or overfitting. For example, a 5 block configuration may be more benefitted by the dropout or the no augmention or the 2 block configuration may benefit from a smaller input size.

Additionally, several low scores are optimisation artefacts, not architectural effects. The five-block network, dropout 0.5 and the two rectangular kernels stopped after 4–21 epochs with validation accuracy of at most 0.61, which suggests unstable training under early stopping with patience 10 on a small validation set. The search ranking of these configurations should therefore be read with caution, since their low scores may reflect unstable training rather than the architecture itself.

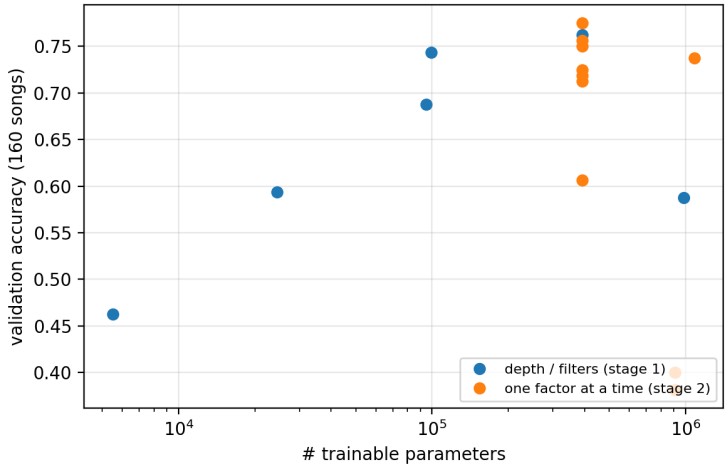


Figure 2: Validation accuracy against number of trainable parameters for the 16 configurations.

Figure 2 plots the validation accuracy against the number of parameters. From this plot, we can see that validation increases with the number of convolutional blocks. With two blocks achieves 0.463, then with 3 0.594 and 0.688, up to 4 blocks. Beyond four blocks the gains are within noise, while the training accuracy of the four-block network is already close to 1, so additional capacity cannot be exploited with 799 training songs. Over-fitting is therefore present in all the larger networks, but it does not explain the dip at five blocks.

3.3 Final Model and Test Results

The configuration with the highest validation accuracy is dropout 0.0 (four blocks, 32–256 filters, augmentation, learning rate 10^{-3}, 128×192 input, Table 3 describes the selected network.), whose best epoch was 39. It is retrained on all 799 training songs for 39 epochs with the same 40-epoch cosine schedule and evaluated once on the 200 test songs. On the main partition it reaches a test accuracy of **0.825** (Wilson 95 % interval 0.766–0.871) with 391,946 parameters, 66.5 s of training time and 8.4 ms of inference per song. Other runs of the same configuration and seed gave between 0.800 and 0.810 (Section 2.3), so the final value should be read as 0.81 ± 0.01 on this partition rather than as an exact figure.

Layer	Output shape	Trainable parameters
Input	$128 \times 192 \times 3$	0
Block 1: Conv 32 + BN + ReLU + MaxPool	$64 \times 96 \times 32$	960
Block 2: Conv 64 + BN + ReLU + MaxPool	$32 \times 48 \times 64$	18,624
Block 3: Conv 128 + BN + ReLU + MaxPool	$16 \times 24 \times 128$	74,112
Block 4: Conv 256 + BN + ReLU + MaxPool	$8 \times 12 \times 256$	295,680
Global average pooling + Dropout	256	0
Dense (10 classes, logits)	10	2,570
Total		**391,946**

Table 3: Selected CNN. All convolutions are 3×3. The dropout rate of the selected configuration is 0.

To measure sensitivity to the partition, the same configuration is retrained on five partitions (Table 4). The accuracy varies between 0.750 and 0.805 (mean 0.772 ± 0.020, mean macro-F1 0.768). This spread (SD 2.0 pp, range 5.5 pp) is of the same order as the 2.8 pp sampling error of a single 200-song test set, so the accuracy of any one partition should not be read as "the" accuracy of the CNN. Training accuracy is close to 1 in every partition (0.995–0.999), so the gap between training and test accuracy is about 22 pp: even with augmentation, the 392 k-parameter network memorises the 799 training songs. The main partition (seed 42) gives the highest of the five accuracies, so it is a relatively favourable partition for the CNN.

Partition (seed)	Test accuracy	Training accuracy
42	0.805	0.999
43	0.770	0.996
44	0.765	0.996
45	0.770	0.997
46	0.750	0.995
Mean ± SD	0.772 ± 0.020	0.997 ± 0.002

Table 4: Selected CNN retrained on five partitions (same hyperparameters; sample standard deviation). The value for seed 42 comes from the multi-partition pipeline and differs by 2 pp from the 0.825 of the Task 1 notebook, which illustrates run-to-run variability.

3.4 Error Analysis of the CNN

To understand how the CNN fails, the test predictions of the five partitions are pooled ($5 \times 200 = 1{,}000$ predictions on 668 distinct songs, since a song can be in the test set of several partitions) and their class probabilities are analysed. The same analysis for the descriptor model and for the fine-tuned VGG16 is summarised in Section 6.4.

Confusion matrix. Figure 3 shows the confusion matrix accumulated over the five partitions, in counts and normalised by row. Classical and metal are the best recognised genres (recall 0.93 and 0.95), while rock is the hardest (recall 0.48, precision 0.61): it is mostly confused with country (11 times), pop (11), metal (9) and disco (7), and country is in turn confused with rock (14 times, the most frequent confusion of the model). Pop is also mixed with disco (10 times) and hip-hop (8), and hip-hop with pop (11). These confusions between neighbouring genres are analysed, and compared with the descriptor-based models, in Section 7.1.2.



Figure 3: Confusion matrix of the CNN trained from scratch, accumulated over the five partitions (1,000 test predictions): counts (left) and fraction of the true genre (right).

Confidence and calibration. The CNN is more confident when it is right (mean top-1 probability 0.87) than when it is wrong (0.62), and the top-1 probability separates correct from wrong predictions well (AUROC 0.84). It is well calibrated (expected calibration error 0.044, Figure 4), with slight over-confidence at intermediate probabilities. Of its 228 errors, 22 (10 %) are made with a top-1 probability of at least 0.90, while 62 % of the correct predictions have such a probability.

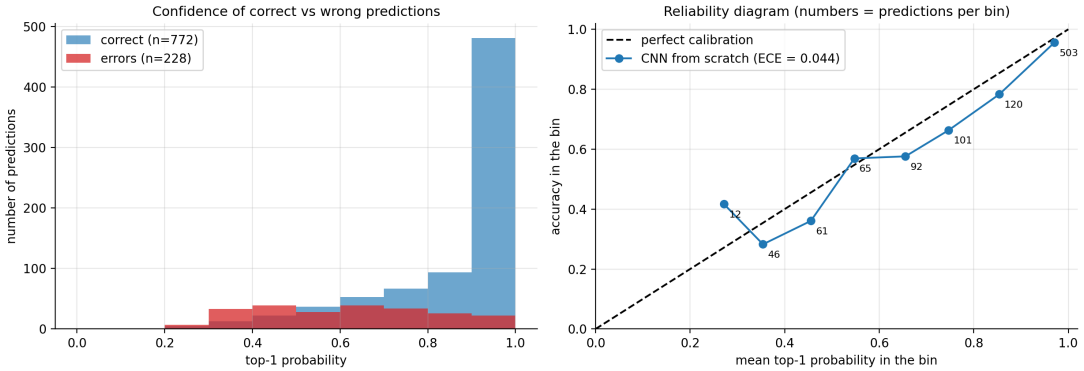


Figure 4: Left: distribution of the top-1 probability of the correct and wrong predictions of the CNN. Right: reliability diagram (numbers: predictions per probability bin).

Types of errors. The correct genre is the first choice in 77.2 % of the predictions, among the first two in 89.4 % and among the first three in 93.8 %. In 54 % of the errors (122 of 228) the true genre was the second most probable class, and in 27 % (62) it was only fourth or worse. Figure 5 classifies the errors: 24 (10.5 %) are near-ties in which the true genre was the runner-up (top-1 and top-2 probabilities within 0.10), another 28 (12.3 %) are near-ties with another genre as runner-up, 86 (37.7 %) are runner-up misses with a clear decision, 68 (29.8 %) have the true genre ranked third or worse, and 22 (9.6 %) are confident errors.

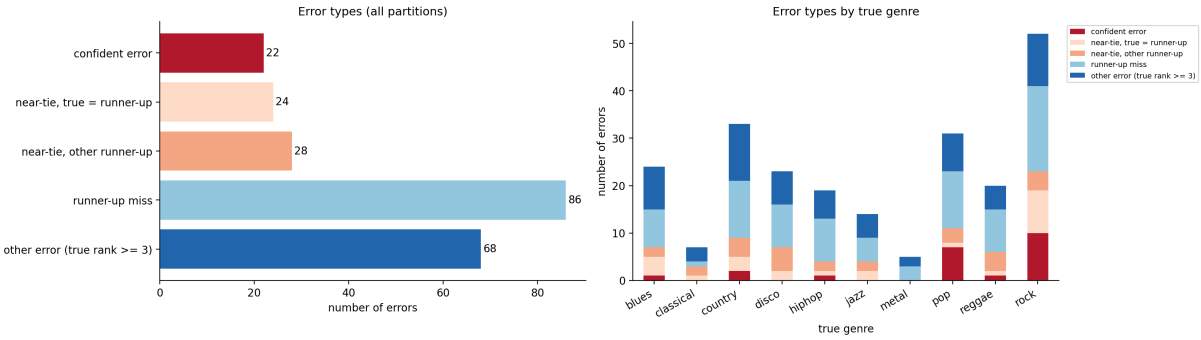


Figure 5: Types of errors of the CNN (all partitions): overall (left) and by true genre (right).

Individual songs. Errors depend strongly on the song: 268 of the 668 songs appear in the test set of at least two partitions, and 46 of them are wrong every time (73 % of the errors on those songs). The song `rock.00051`, for instance, is wrong in the four partitions in which it is evaluated (predicted as pop, with a mean probability of 0.007 for the true genre). The tightest errors are practically coin flips (for example `country.00014` in partition 43: disco and pop both obtain a probability of 0.439, while the true genre is country), whereas the most confident ones reach 0.999 (`rock.00016`, predicted as metal in two partitions).

4 Task 2: Fine-Tuning Pretrained Models

This section studies transfer learning: instead of training a CNN from scratch, networks pretrained on a large dataset are adapted to the genre task. All the experiments of the strategy comparison use the main partition (719 training, 80 validation and 200 test songs, Table 1) and the same preprocessed spectrograms as Task 1; the configuration selected on the validation set is then repeated on five partitions.

4.1 Vision Models (VGG16, ResNet50, EfficientNetB0)

Three architectures pretrained on ImageNet (natural photographs) are used, as provided by Keras. VGG16 is a plain stack of 13 convolutional layers with 3×3 kernels and no normalisation layers. ResNet50 has 50 layers organised in residual blocks with batch normalisation. EfficientNetB0 is a compact network (inverted residual blocks, batch normalisation) designed to give a good accuracy for a small number of parameters. Table 5 summarises them. A spectrogram is not a photograph, but its time–frequency patterns (harmonic lines, onsets, textures) are local structures of the kind that the early convolutional filters of these networks detect, which is the reason to expect that the pretrained weights are a better starting point than random ones. The domain gap is nevertheless large, and it is the reason why adapting the weights, and not only the classifier, is explored.

Model	Layers	Total parameters	Embedding size	Head parameters
VGG16	19	14,719,818	512	5,130
ResNet50	175	23,608,202	2,048	20,490
EfficientNetB0	237	4,062,381	1,280	12,810

Table 5: Pretrained vision models (ImageNet weights) with the classification head used here. "Layers" counts the layers of the convolutional part as listed by Keras, including the input layer.

The spectrograms are cropped and resized to 160×240 pixels (Section 2), keeping the three colour channels of the images, and each network receives the input preprocessing it was trained with (mean subtraction for VGG16 and ResNet50; EfficientNetB0 includes its own rescaling). The convolutional part of each network, without its ImageNet classifier, is followed by global average pooling, a dropout layer (rate 0.3) and a dense layer with 10 softmax outputs. The head therefore has 5,130, 20,490 and 12,810 parameters for VGG16, ResNet50 and EfficientNetB0 respectively.

4.2 Audio Model (VGGish)

As an audio counterpart, totally additionally to the original point of the Lab, VGGish was tested. It is a VGG-like network pretrained on a large collection of audio, which takes the waveform at 16 kHz, computes a log-mel spectrogram with 64 bands and returns one 128-dimensional embedding for every 0.96 s of audio. For each song, the mean and the standard deviation of its embeddings over time are concatenated into a 256-dimensional vector. The TensorFlow Hub version of VGGish cannot be fine-tuned, so it is used as a frozen feature extractor, and three classical classifiers are trained on its embeddings with the training songs: logistic regression, an SVM and a random forest. This experiment was run only on the main partition, with the same training, validation and test songs, and it is exploratory: it is not part of the five-partition comparison.

4.3 Fine-Tuning Strategies

Training the whole network from the start would lead the gradients of a randomly initialised classifier destroy the pretrained filters, so training is done in two phases. In phase 1 the pretrained part is frozen and only the head is trained (Adam, learning rate 10^{-3}). In phase 2 the last layers of the pretrained part are unfrozen and the network is trained with a much smaller learning rate (10^{-5}), so that the filters are adjusted without being overwritten. Each phase lasts at most 30 epochs with early stopping on the validation accuracy (patience 5, best weights restored); the batch size is 32, the loss is the cross-entropy and the images are not augmented. Batch-normalisation layers are kept frozen in every configuration, which is the usual practice with small datasets.

Four strategies are compared for every network (Table 6): training only the head (a linear classifier on frozen features), unfreezing the last convolutional block, the last two blocks, or the whole network. The same four strategies on three networks give 12 configurations.

Model	Head only	Last block	Last two blocks	Full network
VGG16	0	4 (block 5)	8 (blocks 4–5)	19 (all)
ResNet50	0	32 (conv5)	94 (conv4–5)	175 (all)
EfficientNetB0	0	16 (block 7)	75 (blocks 6–7)	237 (all)

Table 6: Number of layers unfrozen in phase 2 for each strategy. Layers are counted as listed by Keras, which is why the counts differ so much between networks.

As an alternative to fine-tuning, the 512-dimensional embeddings of the frozen VGG16 (global average pooling of the last convolutional block) are extracted once for all songs, and logistic regression, an SVM and a random forest are trained on them. This is much cheaper, since the network is never trained, and it answers whether fine-tuning is worth its cost.

The CNN of Task 1 is trained with SpecAugment-style masking, whereas the networks above are not. To check whether this explains part of the differences, the selected configuration is also trained with the same masking (one frequency band and one time band of up to 12.5 % of each axis, filled with the image mean). The configuration is chosen only with the validation accuracy (80 songs, a standard error of about 4.8 pp). The test accuracy of all 12 configurations is reported for completeness, but it is not used to choose. Finally, the selected configuration is retrained on the five partitions of Section 2.2, with an early-stopping validation set taken from each training partition.

4.4 Results

Table 7 gives the results on the main partition and Figure 6 the test accuracy by strategy. Fine-tuning improves over training only the head for the three networks: the best configuration of each network gains 7.5 pp (VGG16, from 0.670 to 0.745), 5.5 pp (ResNet50, from 0.690 to 0.745) and 7.0 pp (EfficientNetB0, from 0.695 to 0.765). The gains are about two standard errors of a 200-song test accuracy (3.2 pp), so they are suggestive but not individually conclusive. The effect of unfreezing more layers is not monotone: for VGG16 the test accuracy rises up to the last two blocks (0.670, 0.705, 0.745) and stays flat for the full network (0.740); for ResNet50 only the full fine-tuning helps (0.690, 0.685, 0.700, 0.745), and for EfficientNetB0 it goes from 0.695 to 0.715 with partial unfreezing and to 0.765 with the full network.

Model	Strategy	Val. acc.	Test acc.	Training time (s)	Inference (ms/song)
VGG16	head only	0.650	0.670	74	3.8
	last block	0.713	0.705	129	4.3
	last two blocks	**0.813**	0.745	161	4.0
	full network	0.775	0.740	251	3.9
ResNet50	head only	0.763	0.690	83	2.8
	last block	0.750	0.685	139	3.0
	last two blocks	0.750	0.700	136	2.9
	full network	0.763	0.745	212	3.1
EfficientNetB0	head only	0.688	0.695	111	1.4
	last block	0.700	0.715	134	1.3
	last two blocks	0.700	0.715	166	1.4
	full network	0.775	**0.765**	223	1.4

Table 7: Results of the 12 fine-tuning configurations on the main partition (80 validation and 200 test songs). The configuration with the best validation accuracy and the one with the best test accuracy are in bold. Training time includes both phases and the first-epoch compilation; the inference time is measured after a warm-up pass.

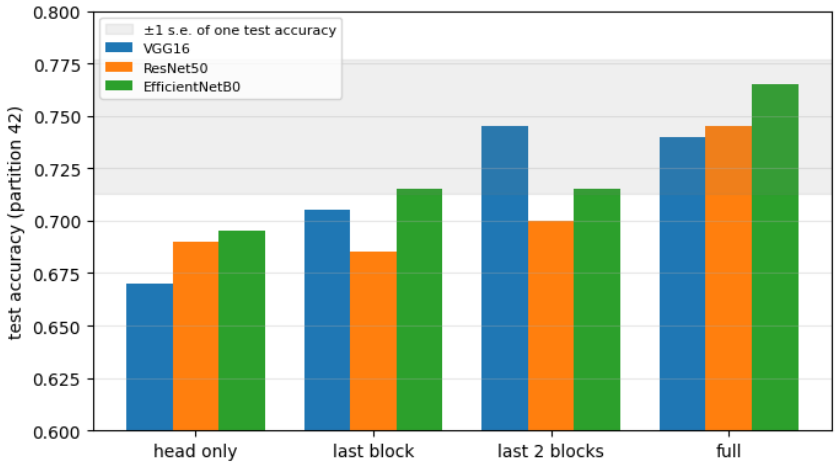


Figure 6: Test accuracy of the 12 configurations on the main partition. The grey band is ±1 standard error of a single test accuracy.

By validation accuracy the best configuration is VGG16 with the last two blocks unfrozen (0.813), whose test accuracy is 0.745 (Wilson 95 % interval $[0.680, 0.800]$), 6.8 pp lower than its validation accuracy. The best test accuracy, 0.765, belongs to EfficientNetB0 fine-tuned entirely, whose validation accuracy is lower (0.775). The validation and test rankings of the 12 configurations are correlated (Spearman 0.64, $p = 0.024$), but the validation set has only 80 songs, so selecting the maximum of 12 noisy values gives an optimistic estimate: the selected configuration is the best on validation partly by chance. This optimism is the main reason why the selected model is later evaluated again on five partitions. At their best, the three networks are indistinguishable on this test set: VGG16 and ResNet50 reach 0.745 and EfficientNetB0 0.765, differences of at most four songs, well inside the sampling error. They differ markedly in cost. EfficientNetB0 has 3.6 times fewer parameters than VGG16 and 5.8 times fewer than ResNet50, and its inference time (1.4 ms per song) is about three times lower than that of VGG16 (4.0 ms) and ResNet50 (3.1 ms). Full fine-tuning takes 2.0 to 3.4 times longer than training only the head, between 212 and 251 s. In terms of accuracy per unit of cost, EfficientNetB0 is therefore the most attractive network here, even though it was not the one selected by validation.

About the frozen embeddings adding a classifier, Logistic regression, an SVM and a random forest on the frozen VGG16 embeddings reach test accuracies of 0.630, 0.685 and 0.690 (validation 0.637, 0.688 and 0.675). This is comparable with training only the head (0.670) and 5.5 to 11.5 pp below the best fine-tuned VGG16 (0.745). Extracting the embeddings of all songs takes about 7 s and training the classifiers 0.5 to 3 s, against 160–250 s of fine-tuning, so this approach is more than an order of magnitude cheaper but clearly less accurate: adapting the convolutional filters to spectrograms is worth its cost. We clearly see that the foundation models have been trained with ImageNet images so they do not transfer well to spectrograms without finetuning.

Training the selected VGG16 configuration with the SpecAugment masking of Task 1 does not improve it: the validation accuracy is 0.763 (against 0.813) and the test accuracy 0.720 (against 0.745). The test difference is five songs and therefore not significant, but it indicates that the masking does not explain the gap between these networks and the CNN trained from scratch (Section 3.3); the optimiser and schedule also differ between the two recipes.

Audio model. The classifiers trained on the frozen VGGish embeddings obtain test accuracies of 0.830 (logistic regression), 0.840 (SVM) and 0.835 (random forest), with validation accuracies of 0.900, 0.913 and 0.900 (Table 8). On the main partition this is 7.5 to 9.5 pp above the best fine-tuned vision model, which is a difference of 15 to 19 songs and therefore much larger than the other differences of this section, and it is also above the CNN of Task 1 (0.805–0.825) and the Lab 1 random forest (0.795), although those differences are within the sampling error. The validation accuracy is about 7 pp above the test accuracy; since the validation set was not used to choose anything in this experiment, the gap reflects the noise of an 80-song set. The comparison with the vision models should be read with caution: it is a single partition, and the pretraining data and model sizes differ. The reported inference time (about 1 s per song) is dominated by reading and resampling the audio file and is not comparable with the image-based times.

Classifier on VGGish embeddings (256-d)	Val. acc.	Test acc.
Logistic regression	0.900	0.830
SVM	0.913	**0.840**
Random forest	0.900	0.835

Table 8: Frozen VGGish embeddings with classical classifiers on the main partition (single partition, exploratory).

Five partitions. Table 9 repeats the selected VGG16 configuration (last two blocks fine-tuned) and the frozen VGG16 with logistic regression on the five partitions, next to the CNN of Task 1. The fine-tuned VGG16 averages 0.677 ± 0.023, about 7 pp below the 0.745 obtained on the main partition: the main-partition value is the optimistic one (best of 12 configurations chosen on 80 validation songs), and the multi-partition pipeline differs slightly from the one used for the search (different validation songs for early stopping and a different image-resizing implementation), which accounts for the 3.5 pp difference between 0.745 and 0.710 on the very same partition. Fine-tuning beats the frozen features by 6.1 pp in every partition (corrected resampled t-test $p = 0.024$), and the CNN trained from scratch beats the fine-tuned VGG16 by 9.5 pp, also in every partition ($p = 0.001$). Pretraining on ImageNet therefore does not help here compared with training a small network on the spectrograms.

Partition (seed)	VGG16 fine-tuned	VGG16 frozen + logistic reg.	CNN from scratch
42	0.710	0.660	0.805
43	0.675	0.585	0.770
44	0.685	0.635	0.765
45	0.650	0.620	0.770
46	0.665	0.580	0.750
Mean ± SD	0.677 ± 0.023	0.616 ± 0.034	0.772 ± 0.020

Table 9: Test accuracy on five partitions of the selected VGG16 configuration, of the frozen VGG16 with logistic regression, and of the CNN of Task 1 (sample standard deviation).

In summary, adapting ImageNet networks to spectrograms works much better than using them as frozen feature extractors, and fine-tuning more of the network generally helps, but the differences between the three architectures and between strategies are mostly within the sampling error of one partition, and the configuration chosen on a small validation set is optimistic. Over five partitions the fine-tuned VGG16 is about 10 pp below the small CNN trained from scratch. Among the pretrained models, the audio model VGGish with a classical classifier is the best on the main partition. The comparison with the CNN and the Lab 1 models is in Section 6.

5 Lab 1 Models Retrained on the Shared Partition

This section retrains the Lab 1 models, which classify songs from hand-crafted descriptors, on exactly the same songs as the Lab 2 models (Section 2.2), so that both approaches can be compared song by song in Section 6 and therefore we can answer the questions. The Lab 1 procedure is reproduced unchanged; only the train/test partition is replaced.

5.1 Descriptors and Lab 1 Pipeline

The Descriptors and Pipeline are the exact same used in Lab1, therefore further explanation about the descriptors, preprocessing,.. is defined in the following public repository: `https://github.com/annacp04/music-genre-classification`. Table 10 lists, for each classifier, the hyperparameter grid and the selection methods tried; the combinations give 21 classifier–selection pairs.

Classifier	Hyperparameter grid	Feature selection tried
Logistic regression	$C \in \{0.01, 0.1, 1\}$	none, ANOVA, PCA, ℓ_1
k-NN	$k \in \{5, 7, 9, 13\}$; uniform or distance weights	none, LDA, PCA, ℓ_1
Decision tree	depth $\in$ {none, 10}; min. leaf $\in \{1, 5\}$; Gini or entropy	none, correlation, RF importance
MLP	hidden layers (128) or $(128, 64)$; L2 $\alpha \in \{10^{-3}, 10^{-2}\}$	none, PCA
Random forest	300 trees, balanced class weights; min. leaf $\in \{1, 3\}$; max. features $\in \{\sqrt{p}, 0.2p\}$	none, correlation
Extra trees	same as random forest	none, correlation
Gradient boosting	100 trees of depth 3; learning rate $\in \{0.05, 0.1\}$; subsample 0.8	none, correlation
AdaBoost	200 estimators; learning rate $\in \{0.1, 0.5\}$; base-tree depth $\in \{2, 3\}$	none
Voting ensemble	soft voting; weights uniform or $(2, 1, 1)$	none

Table 10: Search space of the Lab 1 models. All pipelines start with standardisation.

5.2 Retraining Protocol

On the main partition (seed 42), the grid search of every classifier is run with 10-fold stratified cross-validation (CV) on the 799 training songs only, with accuracy as the criterion. For each classifier–selection pair the best hyperparameters are kept, and the 21 resulting pipelines are ranked by their mean CV accuracy. The whole search took 1,236 s on a Colab CPU (568 s of them for gradient boosting). Only the two selected pipelines are then refitted on the 799 training songs and evaluated, once, on the 200 test songs; the other pipelines are never evaluated on the test set.

The two pipelines with the highest CV accuracy are the random forest with and without the correlation filter, which are essentially one model (the filter removes three of the 58 descriptors). We therefore keep the best of them, random forest with the correlation filter, and add extra trees with the correlation filter, which was the best model of the original Lab 1 and gives continuity with it. This choice only uses CV results, not the test set. The hyperparameters found on the main partition are kept fixed when the two pipelines are retrained on the other four partitions (Section 5.5).

5.3 Model Selection by Cross-Validation

Table 11 gives the best selection method for each classifier (the 21 pipelines are in Appendix A) and Figure 7 shows all of them.

Classifier	Selection	Features	Train acc.	CV acc. ± SD	Macro-F1
Random forest	correlation	55	0.999	**0.785** ± 0.038	0.781
Voting ensemble	none	58	0.999	0.780 ± 0.056	0.778
Extra trees	none	58	0.999	0.779 ± 0.043	0.774
Gradient boosting	none	58	0.999	0.772 ± 0.043	0.771
AdaBoost	none	58	0.974	0.727 ± 0.050	0.732
Logistic regression	none	58	0.843	0.725 ± 0.063	0.721
MLP	PCA	47	0.905	0.724 ± 0.056	0.720
k-NN	LDA	9	0.999	0.706 ± 0.067	0.705
Decision tree	RF importance	50	0.988	0.593 ± 0.046	0.588

Table 11: Best selection method of each classifier, by mean 10-fold CV accuracy on the 799 training songs of the main partition (SD over the ten folds). The extra-trees pipeline with the correlation filter reaches 0.777.

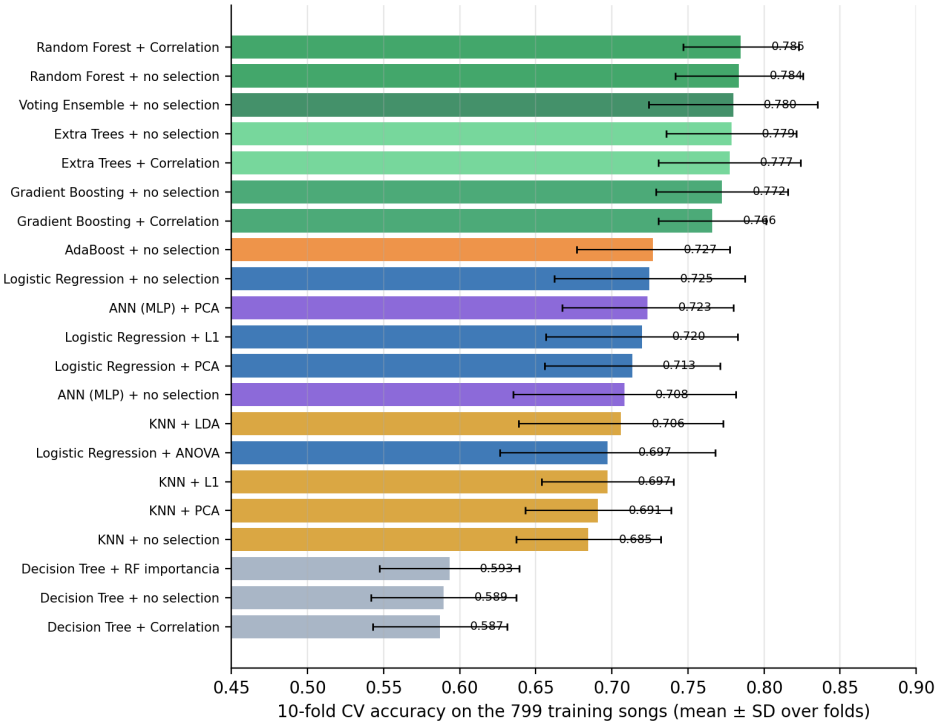


Figure 7: Cross-validation accuracy of the 21 tuned pipelines (main partition, training songs only).

We can see that the tree ensembles lead, random forest, voting ensemble and extra trees all lie between 0.772 and 0.785. AdaBoost, logistic regression, the MLP and k-NN form a second group at 0.71–0.73, and a single decision tree is clearly the worst (0.59). Additionally, we can see that Feature selection matters little for trees and somewhat more for the other models. The correlation filter changes the CV accuracy of the tree ensembles by less than 1 pp. LDA improves k-NN by 2.1 pp (from 0.685 to 0.706), and PCA improves the MLP by 1.5 pp, whereas for logistic regression using all descriptors is best.

5.4 Results

Table 12 reports the two selected pipelines on the 200 test songs of the main partition. Random forest reaches 0.795 and extra trees 0.785 (a difference of two songs), with Wilson 95 % intervals

of about ±5.5 pp. In the original Lab 1, on its own partition, extra trees with the correlation filter obtained a CV accuracy of 0.789 and a test accuracy of 0.800 (macro-F1 0.797); on the shared partition the same pipeline gives 0.777 and 0.785. Both differences are within the sampling error of a 200-song test set, and the fact that random forest and extra trees swap positions is an example of how little separates them.

Pipeline	CV acc.	Test acc.	Wilson 95 % interval	Test macro-F1
Random forest + correlation filter	0.785	0.795	[0.734, 0.845]	0.791
Extra trees + correlation filter	0.777	0.785	[0.723, 0.836]	0.780

Table 12: Selected Lab 1 pipelines on the 200 test songs of the main partition (seed 42).

5.5 Five Partitions

To measure how much the result depends on the partition, the two pipelines are retrained, with the same hyperparameters, on the five partitions used for all models (Table 13). Random forest averages 0.825 ± 0.029 and extra trees 0.809 ± 0.023. Random forest is better than extra trees in all five partitions, by 1.0 to 2.5 pp (mean 1.6 pp; corrected resampled t-test $p = 0.022$), a small but consistent advantage, although with five partitions this test has little power. The main partition is the hardest of the five for both models, and the best partition (seed 43) is 7.5 pp above it for random forest. A conclusion drawn from the main partition alone would therefore understate these models by two to three points relative to their five-partition mean.

Partition (seed)	Random forest + corr.	Extra trees + corr.
42	0.795	0.785
43	0.870	0.845
44	0.835	0.815
45	0.810	0.795
46	0.815	0.805
Mean ± SD	0.825 ± 0.029	0.809 ± 0.023

Table 13: Test accuracy of the selected Lab 1 pipelines on five partitions (sample standard deviation).

In summary, retrained on the shared partition, the Lab 1 models give random forest and extra trees with the correlation filter as the best pipelines, with 0.795 and 0.785 on the main test set and 0.825 ± 0.029 and 0.809 ± 0.023 over five partitions. The ranking among the tree ensembles is not reliable (their cross-validation differences are far below the fold standard deviation), the main partition is the least favourable of the five for them, and the MFCCs are by far the most informative family. Removing the clip length, a non-musical descriptor that leaks the genre, lowers the accuracy to 0.743 ± 0.033 and 0.750 ± 0.027.

6 Comparison of All Models

This section puts all the models of the previous sections side by side. Six models were trained on the five partitions of Section 2.2 and are compared with paired tests: the three descriptor-based pipelines of Lab 1 (random forest with and without the correlation filter, and extra trees with the correlation filter), the CNN trained from scratch, the VGG16 fine-tuned with its last two blocks unfrozen, and the frozen VGG16 with a logistic regression. The other results of Task 2 (the 12 fine-tuning configurations, the other frozen-embedding classifiers and VGGish) exist only for the main partition (seed 42), where they are compared with all the others on the same 200 songs. All the comparisons are made song by song from the stored test predictions of every model, which are analysed in a single notebook. When several pairs are tested at once, Holm's correction is applied to the whole family of pairs (15 pairs for the six models trained on five partitions, 25 comparisons with the best descriptor model for the main-partition comparison); the raw p-values are given when they differ from the corrected ones.

6.1 Five Partitions

Table 14 and Figure 8 give the accuracy of the six models on every partition. The three descriptor-based pipelines lead (0.809 to 0.825 on average), followed by the CNN trained from scratch (0.772 ± 0.020), the fine-tuned VGG16 (0.677 ± 0.023) and the frozen VGG16 with logistic regression (0.616 ± 0.034). The macro-F1 follows the accuracy closely (differences below 0.005 for every model), as expected with balanced classes. The ordering of the approaches (descriptors, CNN, fine-tuned VGG16, frozen VGG16) holds on four partitions out of five; the exception is the main partition (seed 42), where the CNN (0.805) is above the three descriptor pipelines (0.785–0.795). The fine-tuned and the frozen VGG16 are fifth and sixth on every partition (Figure 14).

Model	42	43	44	45	46	Mean ± SD	Macro-F1
Random forest + corr. (Lab 1)	0.795	0.870	0.835	0.810	0.815	0.825 ± 0.029	0.822
Random forest (Lab 1)	0.790	0.865	0.805	0.815	0.800	0.815 ± 0.029	0.813
Extra trees + corr. (Lab 1)	0.785	0.845	0.815	0.795	0.805	0.809 ± 0.023	0.806
CNN from scratch	0.805	0.770	0.765	0.770	0.750	0.772 ± 0.020	0.768
VGG16 fine-tuned	0.710	0.675	0.685	0.650	0.665	0.677 ± 0.023	0.674
VGG16 frozen + log. reg.	0.660	0.585	0.635	0.620	0.580	0.616 ± 0.034	0.617

Table 14: Test accuracy of the six models trained on the five partitions (columns: partition seed), mean ± sample standard deviation and mean macro-F1. The models are sorted by mean accuracy.

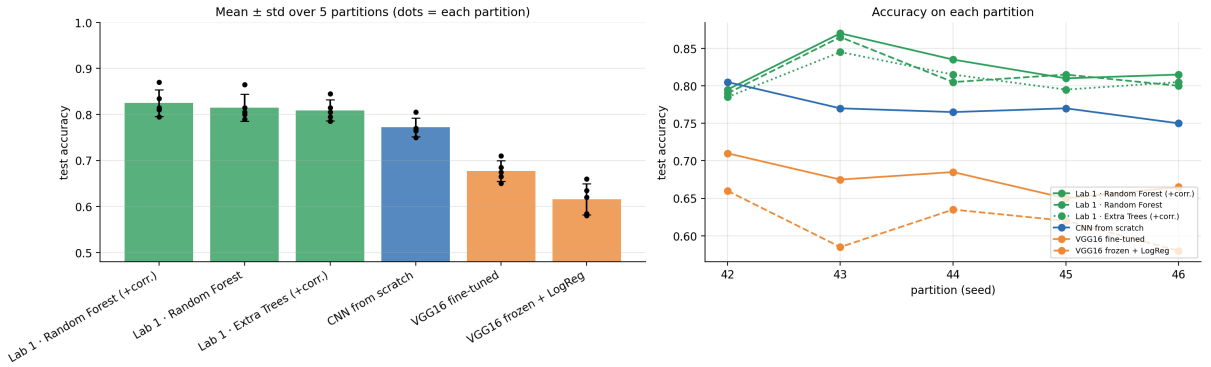


Figure 8: Accuracy of the six models trained on the five partitions. Left: mean ± standard deviation (bars) and accuracy of each partition (dots). Right: accuracy on each partition.

6.2 Main Partition: Every Model on the Same 200 Songs

Figure 9 shows the 26 models evaluated on the 200 test songs of the main partition: the six multi-partition models, the CNN run of Task 1, and the 19 models of Task 2 (the 12 fine-tuning configurations, the fine-tuned VGG16 with SpecAugment, and three classifiers each on frozen VGG16 and frozen VGGish embeddings). The highest accuracies belong to VGGish with a classical classifier (0.830–0.840), the CNN run of Task 1 (0.825) and the CNN of the multi-partition pipeline (0.805), followed by the descriptor pipelines (0.785–0.795). The best fine-tuned vision networks reach 0.745–0.765 and the lowest accuracies correspond to the frozen or head-only VGG16 (0.630–0.690). Every 95 % Wilson interval is 10 to 13 pp wide, so most models overlap with each other. Tested against the best descriptor pipeline (exact McNemar test, Holm correction over 25 comparisons), 23 of the 25 other models are statistically indistinguishable from it; only the frozen VGG16 with logistic regression is significantly worse, in its Task 2 version (0.630, Holm $p = 0.001$) and in its multi-partition version (0.660, Holm $p = 0.018$).

Two pairs of entries of Figure 9 correspond to the same configuration and the same partition: the CNN (0.825 versus 0.805) and the VGG16 with the last two blocks unfrozen (0.745 versus 0.710). The CNN difference is run-to-run variability (Section 2.3). The VGG16 difference reflects mostly the optimism of choosing the best of 12 configurations on an 80-song validation set, together with the different validation songs and image-resizing implementation of the multi-partition pipeline (Section 4.4). The complete list, with intervals and macro-F1, is in Table 26 of the appendix.

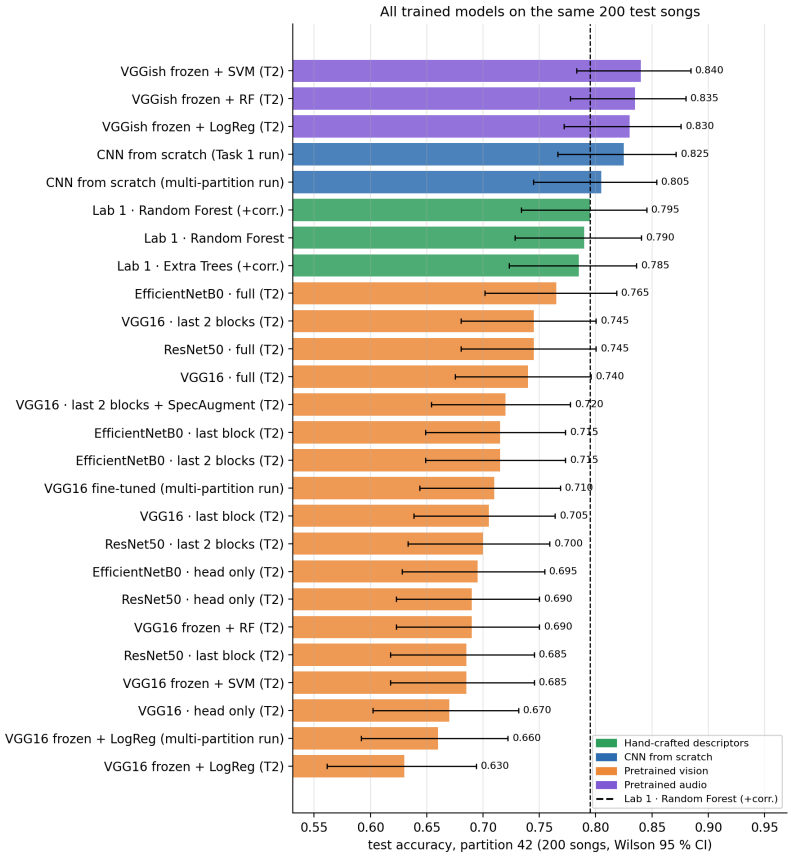


Figure 9: Test accuracy of all the models trained in the project on the 200 test songs of the main partition (partition 42), with Wilson 95 % intervals. The dashed line is the best descriptor pipeline.

6.3 Statistical Comparison of the Models Trained on Five Partitions

Table 15 reports the paired tests for the pairs that matter most for the research questions (all 15 pairs are in Table 27). Two criteria are used together, as explained in Section 2.3: the exact McNemar test on the 200 songs of the main partition and the corrected resampled t-test over the five partitions, both with Holm's correction. A difference is called established only when both tests are significant, the sign agrees, and the winner is better on most partitions.

A vs B	Acc. 42 (A / B)	Only A / only B right	McNemar p (Holm)	Mean diff. (pp)	A wins	t-test p (Holm)	Verdict
RF+corr. vs ET+corr.	0.795 / 0.785	12 / 10	1.000	+1.6	5/5	0.151	n.s.
RF+corr. vs CNN	0.795 / 0.805	15 / 17	1.000	+5.3	4/5	0.637	n.s.
RF+corr. vs VGG16-ft	0.795 / 0.710	35 / 18	0.270	+14.8	5/5	0.065	n.s.
RF+corr. vs VGG16-frozen	0.795 / 0.660	44 / 17	0.010	+20.9	5/5	0.065	partial
CNN vs VGG16-ft	0.805 / 0.710	31 / 12	0.059	+9.5	5/5	0.011	partial
CNN vs VGG16-frozen	0.805 / 0.660	41 / 12	0.001	+15.6	5/5	0.006	different
VGG16-ft vs VGG16-frozen	0.710 / 0.660	24 / 14	1.000	+6.1	5/5	0.151	n.s.

Table 15: Paired comparisons of the models trained on five partitions. "Only A / only B right" counts the songs of the main partition on which exactly one model is correct; the McNemar column is the exact test on that partition. "Mean diff." is the mean accuracy difference A−B over the five partitions, "A wins" the number of partitions in which A is better, and the last test is the corrected resampled t-test of Nadeau and Bengio. Holm's correction is applied over the 15 pairs of the six models; the complete list is in Table 27. Verdict: different = both corrected tests below 0.05; partial = one of them; n.s. = none.

Only one pair meets this rule: the CNN against the frozen VGG16 with logistic regression. The CNN against the fine-tuned VGG16, and the random forest against the frozen VGG16, are significant for one of the two tests only. The remaining differences are not significant after the correction, including the descriptors against the CNN and the three descriptor pipelines among themselves. This does not mean that those models are equivalent: with five partitions and 15 corrected comparisons the tests have little power (Section 7.1.3). It does mean that, apart from the lowest-ranked models, the ranking of the approaches cannot be claimed with confidence from a single partition.

6.4 How the Three Approaches Fail

Besides accuracy, the three approaches (descriptors, CNN from scratch and fine-tuned VGG16) can be compared by the way they fail. For each of them, the test predictions of the five partitions were pooled (1,000 predictions per model) and the class probabilities analysed: confusion matrices, confidence and calibration, the margin between the two most probable genres, and individual songs (the analysis of the CNN is detailed in Section 3.4). Table 16 and Figure 10 summarise the results.

Approach	Top-1	Top-2	Top-3	Conf. correct / wrong	AUROC	ECE	Near-ties (%)	Confident err. (%)	Stable songs (%)
Descriptors (Lab 1)	0.825	0.925	0.972	0.54 / 0.35	0.807	0.320	58	0.0	73
CNN from scratch	0.772	0.894	0.938	0.87 / 0.62	0.838	0.044	23	9.6	73
Pretrained vision (VGG16-ft)	0.677	0.829	0.902	0.85 / 0.64	0.795	0.103	16	14.2	70

Table 16: Error behaviour of the three approaches, from the pooled test predictions of the five partitions (1,000 predictions per model). Conf. = mean top-1 probability; AUROC = ability of the top-1 probability to separate correct from wrong predictions; ECE = expected calibration error; near-ties = errors whose top-1 and top-2 probabilities differ by less than 0.10; confident errors = errors with top-1 probability ≥ 0.90; stable songs = share of the errors on songs (evaluated at least twice) that are wrong every time.

- **Errors are mostly between neighbouring genres.** The true genre is among the two most probable ones in 92.5 % of the predictions of the descriptor model, 89.4 % for the CNN and 82.9 % for VGG16, and in 56 %, 54 % and 47 % of the errors respectively the true genre was the runner-up.
- **The probabilities are not comparable across families.** The random forest probabilities are fractions of tree votes: they are low (mean 0.54 when correct, 0.35 when wrong) and the model is strongly under-confident (ECE 0.320). The networks output softmax probabilities, much more extreme (0.87 and 0.85 when correct, 0.62 and 0.64 when wrong); the CNN is the best calibrated (ECE 0.044) and the fine-tuned VGG16 is over-confident (ECE 0.103). The top-1 probability separates correct from wrong predictions similarly well in the three cases (AUROC 0.81, 0.84 and 0.80).
- **Different types of errors.** 58 % of the errors of the descriptor model are near-ties (top-1 and top-2 probabilities within 0.10) and none is a confident error (top-1 probability of at least 0.90), whereas 9.6 % of the errors of the CNN and 14.2 % of those of VGG16 are confident errors and only 23 % and 16 % are near-ties. The networks therefore tend to be wrong with conviction more often than the forest.
- **Errors are mostly attached to the song.** Among the songs that appear in the test set of at least two partitions, 73 %, 73 % and 70 % of the errors of the three models belong to songs that are wrong every time they are evaluated (46 such songs for the CNN and 33 for the random forest).
- **Genres.** Rock is the hardest genre for the three approaches (recall 0.63, 0.48 and 0.49). Metal and classical are the easiest for the CNN (0.95 and 0.93), while the random forest recognises metal (0.94) and pop (0.93) best; the CNN recognises only 69 % of the pop songs.

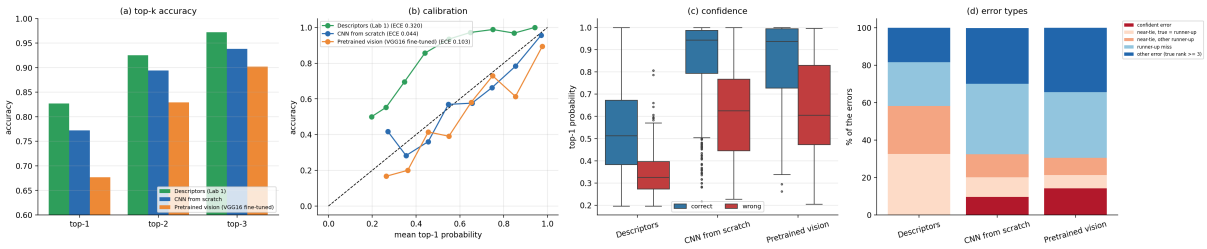


Figure 10: How the three approaches fail (five partitions pooled). (a) Top-k accuracy. (b) Reliability diagrams. (c) Top-1 probability of the correct and wrong predictions. (d) Types of errors as a percentage of the errors.

7 Research Questions

Each answer starts with a short statement followed by the evidence. Unless stated otherwise, the figures come from the comparison of Section 6. The experiments on data size use the first three partitions (seeds 42–44) and a fixed number of epochs, and the test set is used there only to describe how the accuracy changes, never to choose a model.

7.1 Mandatory Questions

7.1.1 Q1. Do learned features beat hand-crafted descriptors?

Answer: No. In this project the learned features do not beat the hand-crafted descriptors; at best they match them.

Over the five partitions, the best descriptor pipeline (random forest with the correlation filter) reaches 0.825 ± 0.029, against 0.772 ± 0.020 for the CNN trained from scratch, 0.677 ± 0.023 for the fine-tuned VGG16 and 0.616 ± 0.034 for the frozen VGG16 (Table 14). The descriptors are ahead of the CNN by 5.3 pp on average (and on four partitions out of five), of the fine-tuned VGG16 by 14.8 pp and of the frozen VGG16 by 20.9 pp (on all five).

The strength of the evidence differs for the three comparisons (Table 15). Against the CNN there is no evidence of a difference: on the main partition the CNN is right on 17 songs where the forest fails and the forest on 15 where the CNN fails (McNemar $p = 0.860$), and the corrected t-test gives $p = 0.127$ (0.637 after Holm). Against the fine-tuned VGG16 the forest is better on every partition, but the evidence is only suggestive once the 15 comparisons are corrected (McNemar on partition 42: $p = 0.027$, Holm 0.270; corrected t-test: $p = 0.005$, Holm 0.065). Against the frozen VGG16, only the McNemar test remains significant after correction ($p = 0.001$, Holm 0.010; t-test $p = 0.005$, Holm 0.065).

On the main partition alone the picture looks different: the CNN (0.805; 0.825 in the Task 1 run) and, above all, the VGGish embeddings with a classical classifier (0.830–0.840) are above the best descriptor pipeline (0.795). None of these differences is significant (McNemar $p = 0.860$ for the CNN, 0.405 for the CNN of Task 1 and 0.188 for VGGish with an SVM, all equal to 1.000 after Holm), and the partition-42 ranking of the CNN is not representative: the main partition is the least favourable of the five for the descriptors and the most favourable for the CNN (Section 7.1.3).

The two families are not uniformly ranked by genre (Figure 11). Compared with the random forest, the CNN has a higher recall for classical music (+6 pp), the same for blues and jazz and marginally higher for metal and reggae (+1 pp), but a much lower recall for pop (−24 pp), rock (−15 pp) and country (−12 pp). The fine-tuned VGG16 is below the random forest on nine genres out of ten (the exception is classical music, +5 pp).

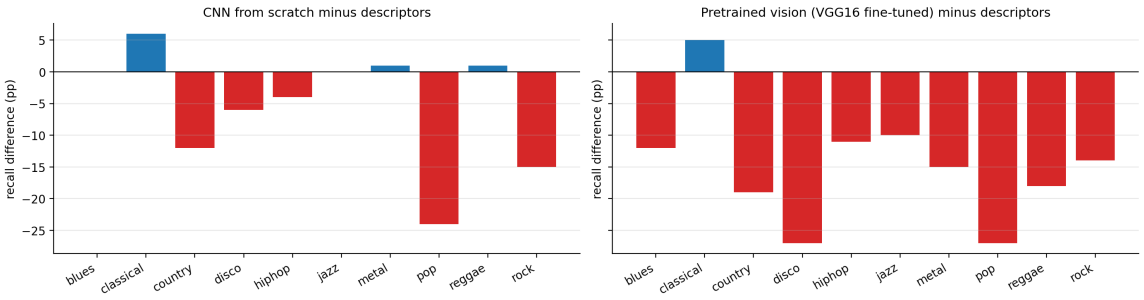


Figure 11: Difference in recall by genre between the CNN (left) or the fine-tuned VGG16 (right) and the best descriptor pipeline, with the five partitions pooled.

Some reasons can be suggested, although they were not isolated experimentally in this work. First, the training set is very small (799 songs) for a network with 392 k parameters, which memorises it (training accuracy 0.997 against 0.772 on the test set), whereas the descriptors, in particular the MFCC statistics, are compact summaries of timbre designed in advance. Second, the CNN sees 8-bit colour-mapped renderings and not the underlying log-mel matrices, so part of the dynamic range is lost. Third, for the pretrained vision networks the domain gap between ImageNet photographs and spectrograms is large (Q9). The learned representations that come closest to the descriptors are the CNN trained from scratch and, on the main partition, the audio embeddings of VGGish, but neither is significantly better.

7.1.2 Q2. Do CNNs and descriptor models make the same mistakes?

Answer: partly. They share a core of difficult songs, but a large part of the errors is specific to each model, and the structure of the genre confusions depends partly on the representation.

Overlap of the errors. Averaged over the five partitions, the random forest makes 35.0 errors on the 200 test songs and the CNN 45.6; 20.4 songs are wrong for both, against 7.9 expected if the errors were independent (Table 17). The correlation between the two error indicators is moderate ($\varphi = 0.39$): the models are not independent (the songs that one gets wrong are harder than average for the other), but 14.6 errors of the forest are correct for the CNN and 25.2 errors of the CNN are correct for the forest. If the best of the two models could be chosen for every song, the accuracy would be 0.898 (oracle), 7.3 pp above the best single model. The fine-tuned VGG16 overlaps even less with the forest ($\varphi = 0.30$), whereas the CNN and the fine-tuned VGG16, which share the same representation, overlap more (35.0 common errors against 14.8 expected, $\varphi = 0.52$), as do the VGG16 with and without fine-tuning ($\varphi = 0.55$). The three descriptor pipelines are, as expected, almost the same model (φ between 0.65 and 0.84 for random forest and extra trees).

Pair (A vs B)	Errors A	Errors B	Both wrong	Expected if indep.	φ	Oracle acc.
RF+corr. vs CNN	35.0	45.6	20.4	7.9	0.39	0.898
RF+corr. vs VGG16-ft	35.0	64.6	22.0	11.3	0.30	0.890
RF+corr. vs VGG16-frozen	35.0	76.8	24.4	13.4	0.30	0.878
CNN vs VGG16-ft	45.6	64.6	35.0	14.8	0.52	0.825
CNN vs VGG16-frozen	45.6	76.8	32.6	17.6	0.37	0.837
VGG16-ft vs VGG16-frozen	64.6	76.8	49.6	24.9	0.55	0.752
RF+corr. vs ET+corr.	35.0	38.2	27.0	6.8	0.68	0.865

Table 17: Overlap between the errors of pairs of models, as the average over the five partitions (200 test songs each). "Expected if indep." is the number of songs that both models would get wrong if their errors were independent; φ is the correlation between the two error indicators; the oracle accuracy is the accuracy obtained by choosing, for every song, whichever of the two models is right.

Shared difficulty. Considering four representative models (the best descriptor pipeline, the CNN, the fine-tuned and the frozen VGG16), 72 pairs (song, partition) out of 1,000 are failed by all four, against 5 expected under independence, and 478 are failed by none, against 266 expected (Figure 12). Difficulty is therefore largely a property of the song: of the errors of the CNN and of the random forest on songs that appear in at least two test sets, 73 % belong to songs that fail every time. On the main partition, 15 songs (7.5 %) are misclassified by all four models: four blues, three disco, four rock, two reggae, one country and one hip-hop song, for instance `blues.00074` (predicted as country by the forest and as disco by the CNN) or `hiphop.00000` (disco for both).

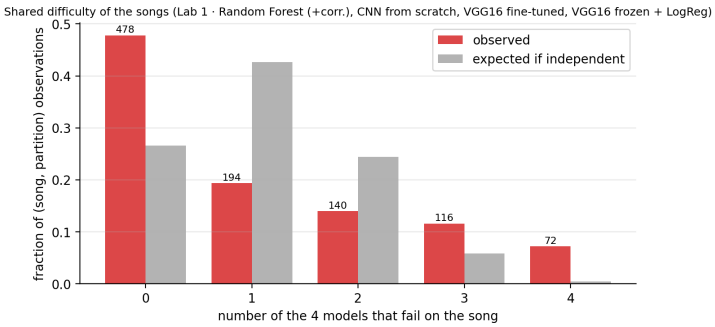


Figure 12: Number of models (among the best descriptor pipeline, the CNN, and the fine-tuned and the frozen VGG16) that fail on each test song, compared with the distribution expected if their errors were independent.

Genre-confusion structure. For each model, the confusion matrices of the five partitions were summed, normalised by row and symmetrised, which gives a similarity between every pair of genres (how often one is taken for the other, Figure 13). The Spearman correlation between the 45 similarities of two models measures whether the same pairs are confused. It is 0.50 between the descriptors and the CNN, 0.53 between the descriptors and the fine-tuned VGG16, and 0.75 between the CNN and the fine-tuned VGG16 (about 0.9 between the fine-tuned and the frozen VGG16). The image-based models therefore agree more with each other than with the descriptors.

Some confusions are common to all the approaches: country–rock is among the three most confused pairs of the random forest (0.07), the CNN (0.12) and VGG16 (0.17), and blues–country

and rock paired with disco, pop or metal appear in several top lists. Other confusions depend on the representation. The descriptor model confuses classical music with jazz (0.080 against 0.035 for the CNN), pop with reggae (0.075 against 0.035) and jazz with rock (0.040 against 0.005); the CNN, on the contrary, confuses pop with disco (0.080 against 0.005 for the descriptors), with rock (0.070 against 0.000) and with hip-hop (0.095 against 0.030), and also classical with rock (0.050 against 0.005). A plausible reading, not tested here, is that global statistics of timbre (MFCC means and variances) make acoustic genres such as classical and jazz look alike, while the time–frequency patterns used by the CNN separate them but mix the popular genres. The structure found with the descriptors in Lab 1 is therefore only partly the structure of the genres themselves: part of it is an effect of the descriptors.

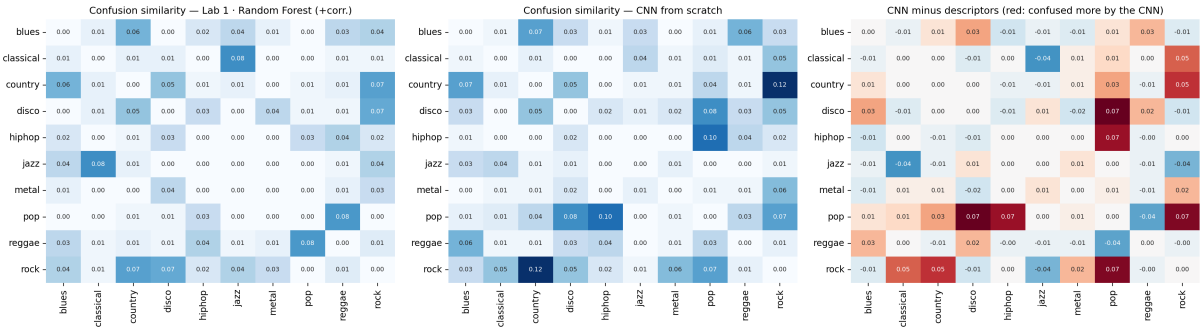


Figure 13: Similarity between genres (symmetrised confusion, five partitions pooled) for the descriptor pipeline (left) and the CNN (centre), and their difference (right; red: confused more often by the CNN).

7.1.3 Q5. How much does one 80/20 split tell you?

Answer: little. The accuracy of a single partition moves by several points just through the choice of the test songs, it can reverse the ranking of two models, and only differences of about 7 pp or more can be detected with it.

Spread across partitions. The accuracy of every model changes by 5.5 to 8.0 pp between its best and its worst partition (standard deviation 2.0 to 3.4 pp, Table 18). This spread is of the size expected from sampling alone: the ratio between the observed standard deviation and the binomial standard error of a 200-song test set is between 0.68 and 1.08, so most of the variation is noise in the choice of the 200 test songs and not instability of the training.

Model	Mean acc.	Observed SD (pp)	Binomial SE (pp)	Ratio	Range (pp)	Best / worst partition
Random forest + corr. (Lab 1)	0.825	2.9	2.7	1.08	7.5	43 / 42
Random forest (Lab 1)	0.815	2.9	2.7	1.07	7.5	43 / 42
Extra trees + corr. (Lab 1)	0.809	2.3	2.8	0.83	6.0	43 / 42
CNN from scratch	0.772	2.0	3.0	0.68	5.5	42 / 46
VGG16 fine-tuned	0.677	2.3	3.3	0.68	6.0	42 / 45
VGG16 frozen + log. reg.	0.616	3.4	3.4	0.98	8.0	42 / 46

Table 18: Spread of the accuracy across the five partitions compared with the standard error expected from the sampling noise of a 200-song test set alone (binomial standard error at the mean accuracy of each model).

The ranking depends on the partition. The main partition (seed 42) is the worst of the five for the three descriptor pipelines (0.785–0.795, against 0.870 at best on partition 43) and the

best for the CNN, the fine-tuned VGG16 and the frozen VGG16. As a consequence, the CNN is first on the main partition and fourth on the four others (Figure 14): a conclusion drawn from the main partition alone ("the CNN is better than the descriptors, 0.805 against 0.795") would be reversed by the five-partition mean (0.772 against 0.825). The same configuration trained twice on the same partition also changes by a few songs: the CNN gave 0.825 in the Task 1 notebook and 0.805 in the multi-partition pipeline.

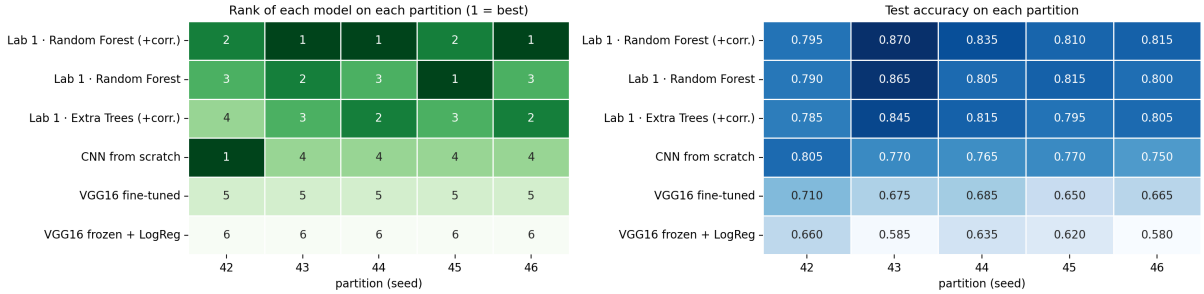


Figure 14: Rank (left) and accuracy (right) of every model on each partition. The CNN is first on the main partition (42) and fourth on the others.

What can be detected. With 200 songs, one song is 0.5 pp. In this study two different models disagree on 13 to 65 of the 200 songs (median 43). With 40 discordant songs, an exact McNemar test needs a net advantage of 14 songs (7 pp) to be significant at the 5 % level, and 8 to 9 pp with 50 to 60 discordant songs (Table 19); this is before any correction for multiple comparisons. The corrected resampled t-test over five partitions is also conservative: its variance factor is 0.45 instead of the 0.20 of a naive paired test, which divides the t statistic by 1.5, and with four degrees of freedom it requires $|t| > 2.78$. Figure 15 shows the paired differences between the key models on each partition.

Discordant songs	Songs won / lost by the better model	Net advantage (songs)	Difference (pp)
10	9 / 1	8	4.0
20	15 / 5	10	5.0
30	21 / 9	12	6.0
40	27 / 13	14	7.0
50	33 / 17	16	8.0
60	39 / 21	18	9.0

Table 19: Smallest advantage of one model over another that is significant with an exact two-sided McNemar test at the 5 % level on 200 songs, as a function of the number of songs on which the two models disagree (before any multiple-comparison correction).

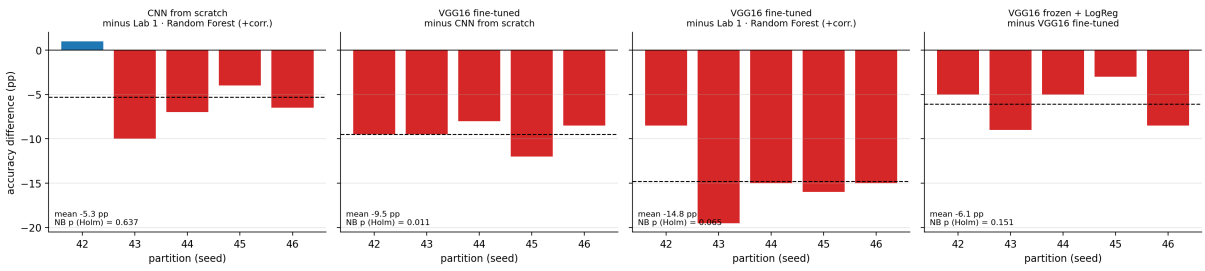


Figure 15: Paired accuracy difference between pairs of models on each partition (bars), with its mean (dashed line) and the corrected t-test p-value (Holm).

Consequences for the report. Differences between two models are claimed only when both tests (McNemar with Holm and corrected t-test with Holm) agree and the winner is better on most partitions; with this rule, only the CNN against the frozen VGG16 is established (Table 15). In all other cases the models are said to tie within the margin of error, quoting the spread over the partitions and the number of discordant songs.

7.2 Additional Questions

7.2.1 Q3. Are the two approaches complementary?

Answer: yes, partially. The descriptors and the CNN make different errors and a well-scaled combination of their probabilities is better than the best of the two on all five partitions, although the gain (about 3 pp) is borderline once multiple comparisons are accounted for.

The ceiling is given by the oracle accuracy: choosing, for every song, whichever of the two models is right would give 0.898 for descriptors and CNN, against 0.825 for the best single model (Section 7.1.2). To approach it, the class probabilities of the models are averaged (soft voting) for every subset of four representative models (the random forest with the correlation filter, the CNN, the fine-tuned VGG16 and the frozen VGG16), on the five partitions and with the same test songs for all (Table 20, Figure 16).

A plain average does not work: descriptors plus CNN reach 0.813, 1.2 pp *below* the forest alone. The reason is a mismatch of scales (Section 6.4): the forest outputs vote fractions (mean top-1 probability about 0.5) and the networks output values close to 0 or 1 (mean top-1 probability about 0.8), so the average is dominated by the networks whatever their quality. Matching the confidence of the models before averaging fixes this. Each model's probabilities are raised to a power (and renormalised) so that its mean top-1 probability equals a common value (0.70). This procedure uses only the predicted probabilities, no labels, and the value 0.70 was fixed beforehand; the accuracy of the combination changes by less than 1 pp for any target between 0.4 and 0.9 (Figure 17).

With matched confidences, the combination of descriptors and CNN reaches 0.854 ± 0.018, against 0.825 for the random forest alone (+2.9 pp), and it is better on the five partitions. On the main partition the ensemble is right on 18 songs where the forest fails and wrong on 4 where the forest is right (McNemar $p = 0.004$); the corrected t-test gives $p = 0.154$. Both values become non-significant after Holm's correction over the 22 ensembles evaluated (0.096 and 1.000), so the result should be described as consistent and in the expected direction, but not established beyond doubt. Song by song, among the 199 test cases (summed over the partitions) where exactly one of the two models is right, the ensemble chooses the right answer in 152 (76 %); it never breaks a song on which both are right (0 of 699), and it rescues 3 of the 102 songs that both get wrong.

Ensemble	Prob.	Accuracy (5 part.)	Best member	Gain (pp)	Wins	McNemar p (Holm)	t-test p (Holm)	Oracle
RF+corr. + CNN	matched	0.854 ± 0.018	0.825	+2.9	5/5	0.004 (0.096)	0.154 (1.000)	0.898
RF+corr. + CNN	raw	0.813 ± 0.020	0.825	−1.2	1/5	0.136 (1.000)	0.654 (1.000)	0.898
RF+corr. + CNN + VGG16-ft	matched	0.839 ± 0.022	0.825	+1.4	3/5	0.023 (0.453)	0.518 (1.000)	0.914
RF+corr. + CNN + VGG16-frozen	matched	0.826 ± 0.034	0.825	+0.1	2/5	0.043 (0.813)	0.973 (1.000)	0.919
RF+corr. + VGG16-ft	matched	0.819 ± 0.017	0.825	−0.6	2/5	0.327 (1.000)	0.761 (1.000)	0.890
RF+corr. + CNN + VGG16-ft + VGG16-frozen	matched	0.803 ± 0.036	0.825	−2.2	1/5	0.150 (1.000)	0.469 (1.000)	0.928
CNN + VGG16-ft	matched	0.768 ± 0.024	0.772	−0.4	1/5	1.000 (1.000)	0.466 (1.000)	0.825

Table 20: Soft-voting ensembles on the five partitions (selected rows of the 22 evaluated). "matched" = confidence-matched average, "raw" = plain average of the stored probabilities. The gain is measured against the best single member (mean accuracy over the five partitions); McNemar is computed on the main partition; Holm's correction is applied over all 22 ensembles.

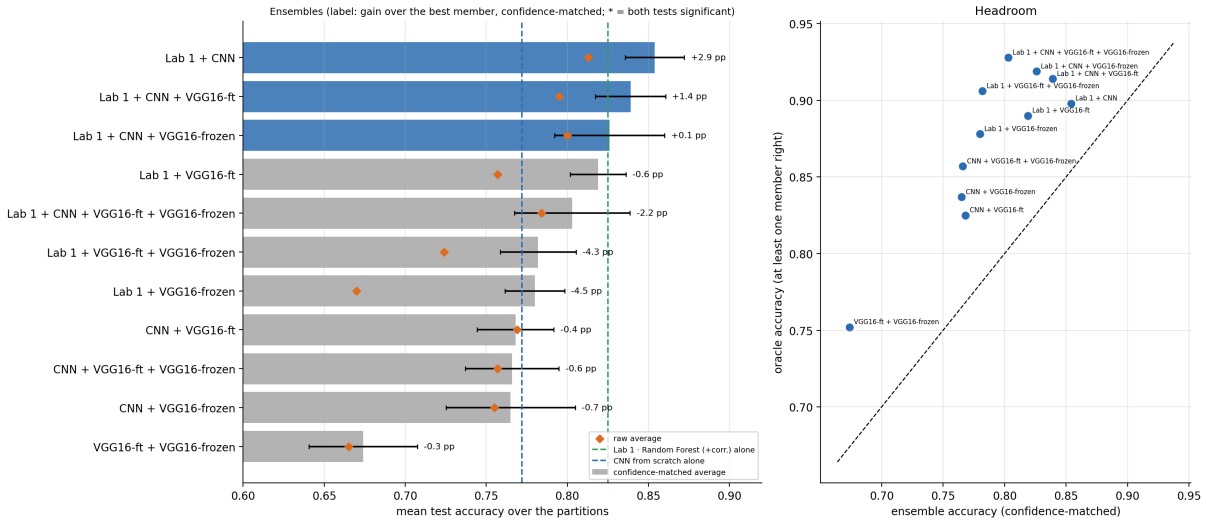


Figure 16: Left: accuracy of the soft-voting ensembles with confidence-matched probabilities (bars, mean ± SD over the five partitions; diamonds: plain average) against the best single models (dashed lines). Right: accuracy against oracle accuracy (labels: members).

Adding more models does not help. The three-model combination with the fine-tuned VGG16 reaches 0.839 (+1.4 pp), with the frozen VGG16 0.826 (+0.1 pp), and the four-model combination 0.803, below the forest alone, although its oracle accuracy is the highest (0.928). The weaker models are less accurate and their errors overlap with those of the CNN ($\varphi = 0.52$ between CNN and fine-tuned VGG16), so the extra information is smaller than the extra noise. Combinations that do not include the descriptors never improve on their best member. The weight given to each model matters little within a wide range: the combination of descriptors and CNN is above the random forest alone for descriptor weights between roughly 0.35 and 0.95, with a maximum of about 0.865 around 0.6–0.7. This curve is descriptive and was not used to choose the reported ensemble (Figure 17).

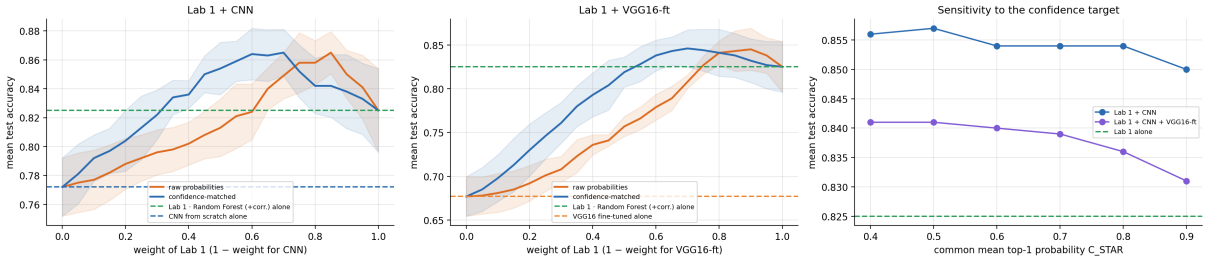


Figure 17: Accuracy of the combination of the descriptor model with the CNN (left) and with the fine-tuned VGG16 (centre) as a function of the weight of the descriptor model, for raw and confidence-matched probabilities; right: sensitivity to the confidence target. Descriptive curves, not used to choose the ensemble.

7.2.2 Q4. Is the extra cost worth it?

Answer: no, not for accuracy. The random forest on descriptors is both the most accurate and one of the cheapest models; the CNN costs about 22 times more in training time and inference time for a lower accuracy.

Table 21 and Figure 18 compare the accuracy with the training time, the inference time and the size of the models. The random forest needs 2.7 s of training and 0.19 ms per song and occupies 15.3 MB. The CNN needs 58.7 s (21.5 times more) and 4.26 ms per song (22 times more) for an accuracy that is 5.3 pp lower (no significant difference, Q1); only in model size is it smaller (1.57 MB, about a tenth, because the forest stores 300 deep trees). Fine-tuning VGG16 takes 127 s (47 times more), 8.1 ms per song and 58.9 MB (3.9 times more) with 14.8 pp less accuracy, and the frozen VGG16 trains in 0.35 s but needs 12 ms per song to compute the embedding. In the accuracy–training-time plane, the models that are not dominated (no other model is both cheaper and at least as accurate) are the frozen VGG16 (the cheapest), the extra trees and the random forest with the correlation filter; the CNN and the fine-tuned VGG16 are dominated, as they are slower and less accurate. In the main-partition experiments of Task 2, fine-tuning the three networks takes between 129 and 251 s, EfficientNetB0 being the cheapest at inference (1.4 ms per song), and VGGish needs about one second per song, dominated by reading and resampling the audio (Table 24).

Model	Mean acc.	Training (s)	Inference (ms/song)	Size (MB)	Parameters
Random forest + corr. (Lab 1)	0.825	2.7 (1.0×)	0.19 (1.0×)	15.29 (1.00×)	–
Random forest (Lab 1)	0.815	3.4 (1.2×)	0.19 (1.0×)	13.95 (0.91×)	–
Extra trees + corr. (Lab 1)	0.809	1.1 (0.4×)	0.24 (1.2×)	35.76 (2.34×)	–
CNN from scratch	0.772	58.7 (21.5×)	4.26 (22.0×)	1.57 (0.10×)	391,946
VGG16 fine-tuned	0.677	127.3 (46.7×)	8.07 (41.7×)	58.88 (3.85×)	14,719,818
VGG16 frozen + log. reg.	0.616	0.4 (0.1×)	12.02 (62.1×)	0.02 (0.00×)	5,130

Table 21: Accuracy and cost of the six models trained on five partitions (mean over the partitions). In brackets, the cost relative to the random forest with the correlation filter. The inference time of the VGG16 models includes the forward pass through the network; that of the frozen VGG16 includes extracting the embedding.

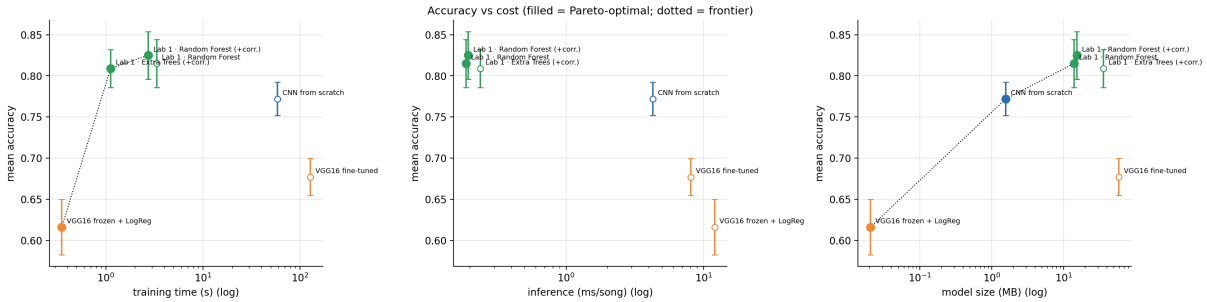


Figure 18: Accuracy against training time, inference time and model size (logarithmic axes) for the six models trained on five partitions. Filled markers: Pareto-optimal models; dotted line: Pareto frontier.

Two caveats limit this comparison. The training times were measured on different hardware (the forests on a CPU, the networks on a Colab GPU), and the cost of obtaining the inputs is not included: the descriptors must be extracted from the audio, and the spectrograms rendered as images. The only extra cost that buys accuracy in this study is the combination of descriptors and CNN (Q3): it requires both models (about 61 s of training and 4.5 ms per song) for +2.9 pp over the forest alone.

7.2.3 Q6. How does data size affect the comparison?

Answer: the descriptor model is much better with little data, and the gap shrinks as the amount of data grows, but the CNN curve is confounded with the number of optimisation steps and the data are insufficient to extrapolate.

Table 22 and Figure 19 show the accuracy of the CNN and of the best Lab 1 model trained on 10, 25, 50 and 100 % of the training songs (79, 199, 399 and 799 songs, stratified by genre), always tested on the same 200 songs. With 10 % of the data the random forest keeps 66 % of its full accuracy (0.553 against 0.833), whereas the CNN keeps 26 % (0.207 against 0.782). The gap in favour of the descriptors is 34.7 and 35.5 pp at 10 and 25 %, 14.8 pp at 50 % and 5.2 pp with all the data. The forest flattens (+6.6 pp from 50 to 100 %), while the CNN is still rising steeply (+16.4 pp), so the CNN would probably benefit more from additional data; however, nothing is observed beyond 799 songs.

Training songs (%)	Songs	CNN from scratch	Lab 1 (random forest + corr.)	Gap (pp)
10	79	0.207 ± 0.077	0.553 ± 0.037	34.7
25	199	0.333 ± 0.045	0.688 ± 0.012	35.5
50	399	0.618 ± 0.076	0.767 ± 0.049	14.8
100	799	0.782 ± 0.028	0.833 ± 0.038	5.2

Table 22: Test accuracy against the amount of training data (mean ± sample SD over the partitions 42–44; same 200 test songs per partition). The gap is Lab 1 minus CNN.

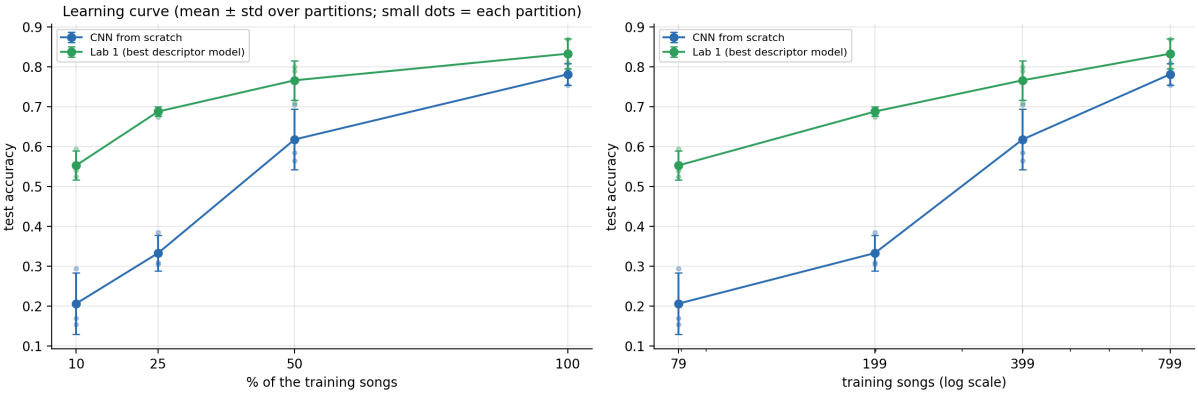


Figure 19: Learning curves of the CNN and of the best Lab 1 model as a function of the percentage (left) and the number (right) of training songs; mean ± SD over partitions 42–44 (small dots: each partition).

The comparison at small sizes must be read with care. The number of epochs of the CNN is fixed (39), so with 10 % of the data the network performs about 120 weight updates instead of about 980; at 10–25 % it is therefore partly *under-trained*, and its curve mixes the effect of having less data with that of having fewer training steps. The random forest has no such limitation. The results are also averaged over only three partitions (the 100 % points, 0.782 and 0.833, differ slightly from the five-partition means, 0.772 and 0.825) and the standard deviation across partitions is large for small fractions.

7.2.4 Q9. Would fine-tuning a pretrained model improve the model?

Answer: fine-tuning clearly improves a pretrained vision network over using it frozen, but fine-tuned ImageNet networks stay below the CNN trained from scratch and the descriptors. The audio model VGGish is a better starting point than any vision model, although the evidence for it comes from a single partition.

Fine-tuning against the frozen network. On the main partition, unfreezing layers improves the best configuration of each network by 7.5 pp for VGG16 (0.670 to 0.745), 5.5 pp for ResNet50 (0.690 to 0.745) and 7.0 pp for EfficientNetB0 (0.695 to 0.765) (Section 4.4). Each of these differences is individually suggestive (McNemar $p = 0.040$, 0.027 and 0.007; 0.053, 0.053 and 0.020 after Holm over the three networks). Over five partitions, the fine-tuned VGG16 is 6.1 pp better than the frozen VGG16 with logistic regression on every partition (corrected t-test $p = 0.024$, 0.151 after Holm over 15 pairs; McNemar on partition 42: $p = 0.143$).

Fine-tuned vision against the other approaches. The benefit is not enough to reach the other approaches. The CNN trained from scratch is 9.5 pp better than the fine-tuned VGG16 on every partition (corrected t-test $p = 0.001$, 0.011 after Holm; McNemar on partition 42 $p = 0.005$, 0.059 after Holm), and the descriptors are 14.8 pp better (Table 15). The best of the 12 configurations on the main partition (EfficientNetB0 fully fine-tuned, 0.765) is not significantly different from the CNN or the descriptors (Table 23), but this value is the maximum of 12 estimates and optimistic: the same VGG16 configuration drops from 0.745 to 0.710 on the same partition in the multi-partition pipeline, and to 0.677 on average over five partitions. ImageNet pretraining therefore does not help here compared with training a small network on the spectrograms. The large domain gap and the size of the networks (14.7 million parameters for VGG16, about 38 times those of the CNN) for 799 training songs are the plausible reasons.

Vision against audio. Pretrained on audio and used as a frozen extractor, VGGish with a classical classifier reaches 0.830 (logistic regression), 0.840 (SVM) and 0.835 (random forest) on the main partition. With the same classifiers, the frozen VGG16 gets 0.630, 0.685 and 0.690: VGGish is 14.5 to 20.0 pp better, and the difference is highly significant for each classifier (McNemar $p < 10^{-4}$). Against the best fine-tuned vision networks the advantage is 8.5 to 9.5 pp for VGG16 (McNemar $p = 0.008$ to 0.019) and 6.5 to 7.5 pp for EfficientNetB0 ($p = 0.028$ to 0.060), but these differences do not survive the Holm correction over 28 comparisons (Table 23). The audio model is nominally above the CNN and the descriptors (0.840 against 0.805 and 0.795), without significant difference. The audio embeddings are therefore the best pretrained starting point of this study, but one partition and 200 songs only allow this to be stated with moderate confidence. Part of the advantage may also be due to the input: VGGish works on the waveform and not on 8-bit colour-mapped images, and this effect cannot be separated here from that of the pretraining domain. CLAP, mentioned in the question, was not tested.

VGGish frozen + SVM versus	Acc. (VGGish / other)	Only VGGish / only other right	McNemar p	Holm p
VGG16 fine-tuned, last 2 blocks	0.840 / 0.745	33 / 14	0.008	0.183
EfficientNetB0 fine-tuned, full	0.840 / 0.765	28 / 13	0.028	0.523
VGG16 frozen + SVM	0.840 / 0.685	38 / 7	<0.001	<0.001
CNN from scratch	0.840 / 0.805	22 / 15	0.324	1.000
Random forest + corr. (Lab 1)	0.840 / 0.795	23 / 14	0.188	1.000

Table 23: Exact McNemar tests between VGGish with an SVM (the best audio setting) and other models on the 200 songs of the main partition. Holm's correction is applied over the 28 pairs among the eight models of this comparison (the three VGGish classifiers, the fine-tuned VGG16 and EfficientNetB0, the frozen VGG16 with an SVM, the CNN and the random forest).

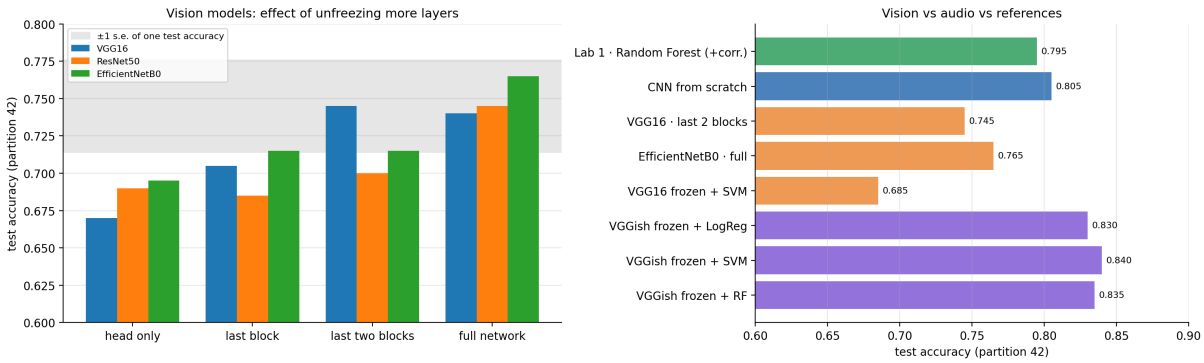


Figure 20: Left: test accuracy of the 12 fine-tuning configurations of Task 2 (partition 42; band: ±1 standard error of one test accuracy). Right: vision and audio pretrained models compared with the CNN and the descriptor model.

7.2.5 Q10. Frozen embeddings + traditional ML algorithm, or full fine-tuning?

Answer: it depends on how close the pretraining domain is to the task. With VGGish (audio), frozen embeddings and a classical classifier are the best option, and extremely cheap to train; with ImageNet networks, fine-tuning is clearly better than frozen embeddings, at a much higher cost, but still falls short of the alternatives.

Table 24 and Figure 21 give the accuracy and the cost of both strategies on the main partition. For VGG16, the classifiers trained on the frozen embeddings reach 0.630 (logistic regression), 0.685 (SVM) and 0.690 (random forest), which is similar to training only a linear head (0.670)

and 5.5 to 11.5 pp below the fine-tuned network with the last two blocks unfrozen (0.745). The difference is significant against logistic regression (McNemar $p = 0.0014$, Holm 0.004) but not against the SVM and the random forest ($p = 0.081$ and 0.099, Holm 0.161). The classifiers train in 0.5–3.0 s, against 161 s for fine-tuning: fine-tuning is 50 to 320 times more expensive for a gain of 5.5 to 11.5 pp. Extracting the embeddings of the 999 songs takes about 7 s. Over five partitions, the fine-tuned VGG16 is 6.1 pp above the frozen VGG16 with logistic regression on every partition (t-test $p = 0.024$, Holm 0.151), and its inference is cheaper in the multi-partition pipeline (8.1 ms against 12.0 ms per song, since the frozen configuration also pays for the embedding).

With VGGish, the situation is reversed: the frozen embeddings with an SVM reach 0.840 with 0.4 s of training, above any fine-tuned ImageNet network. Their cost is concentrated in the embedding, about one second per song, dominated by reading and resampling the audio. VGGish cannot be fine-tuned in its TensorFlow Hub version, so full fine-tuning of an audio model could not be compared.

Setting	Val. acc.	Test acc.	Training (s)	Inference (ms/song)
VGG16 frozen + LogReg	0.638	0.630	0.5	7.2
VGG16 frozen + SVM	0.688	0.685	1.0	7.8
VGG16 frozen + RF	0.675	0.690	3.0	7.4
VGG16 head only (frozen features, linear layer)	0.650	0.670	73.5	3.8
VGG16 last 2 blocks (fine-tuned, selected)	0.812	0.745	160.5	4.0
VGG16 full network	0.775	0.740	251.2	3.9
VGGish frozen + LogReg	0.900	0.830	0.6	1002.0
VGGish frozen + SVM	0.912	0.840	0.4	1002.1
VGGish frozen + RF	0.900	0.835	3.0	1002.1

Table 24: Frozen embeddings with classical classifiers against fine-tuning, on the main partition (Task 2 runs). Training time of the frozen settings excludes the extraction of the embeddings (about 7 s for all songs with VGG16, about 1,000 s with VGGish); the inference time of the frozen settings includes it.

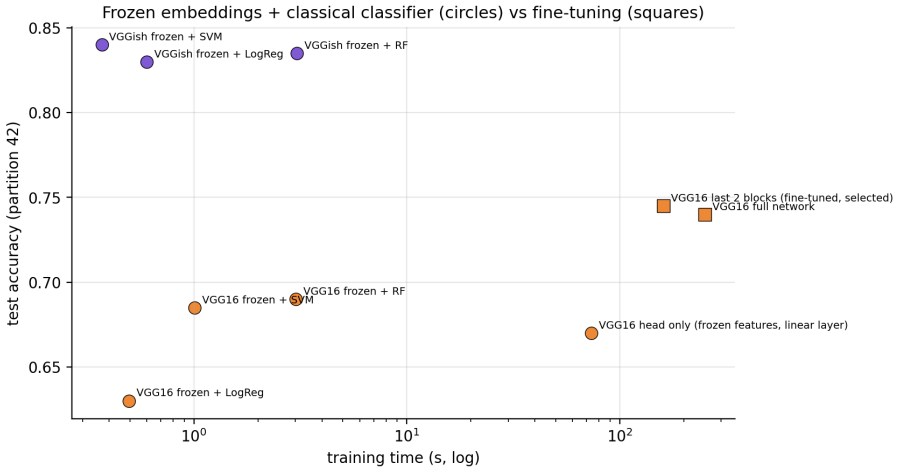


Figure 21: Accuracy against training time of frozen embeddings with classical classifiers (circles) and fine-tuned networks (squares) on the main partition.

8 Conclusions

All the models of the lab (hand-crafted descriptors, a CNN trained from scratch and pretrained networks) were trained and tested on the same songs, on five partitions, and compared with paired tests. The main findings are:

1. **Learned features do not beat the descriptors (Q1).** Over five partitions the best descriptor pipeline (random forest with a correlation filter) reaches 0.825 ± 0.029, against 0.772 ± 0.020 for the CNN from scratch, 0.677 ± 0.023 for the fine-tuned VGG16 and 0.616 ± 0.034 for the frozen VGG16. The difference with the CNN (5.3 pp) is not significant; the descriptors are better than the frozen VGG16 (significant on one of the two tests) and nominally better than the fine-tuned VGG16 (not significant after correction). Only on the main partition, and without significance, do the CNN (0.805) and VGGish embeddings (0.830–0.840) come out above the descriptors (0.795).

2. **A single split is not reliable (Q5).** The accuracy moves by 5.5 to 8.0 pp between partitions, which is the size expected from the sampling noise of 200 songs. The main partition is the worst for the descriptors and the best for the CNN, which ranks first there and fourth on the others. On a single partition only differences of about 7 pp or more can be detected; the only difference established by both tests is that between the CNN and the frozen VGG16.

3. **The errors are partly shared and partly model-specific (Q2).** Two models fail together on about two to two and a half times as many songs as expected by chance, and 72 song–partition pairs are failed by four models against 5 expected, so difficulty belongs largely to the song. But the descriptors and the CNN still differ on about 40 songs per partition, and they confuse different pairs of genres (classical–jazz and pop–reggae for the descriptors, pop with disco, rock and hip-hop for the CNN; Spearman correlation of the confusion structures 0.50).

4. **They are complementary (Q3).** A confidence-matched average of the descriptor model and the CNN reaches 0.854 ± 0.018 (+2.9 pp over the best single model, better on all five partitions), whereas a plain average of the probabilities fails because the probability scales of the forest and of the networks differ. The gain is consistent but borderline in statistical terms after correcting for multiple comparisons.

5. **The extra cost is not worth it for accuracy alone (Q4).** The random forest trains in 2.7 s and classifies in 0.19 ms per song; the CNN needs about 22 times more time for a lower accuracy, and the fine-tuned VGG16 about 47 times more.

6. **Data size (Q6).** With 10 % of the training songs the descriptor model keeps 66 % of its accuracy and the CNN 26 %; the gap closes from 35 pp to 5 pp with all the data, but the CNN curve is confounded with the number of optimisation steps.

7. **Pretraining (Q9, Q10).** Fine-tuning improves ImageNet networks over frozen features by 5.5–7.5 pp, but fine-tuned VGG16 stays 9.5 pp below the CNN from scratch. The audio model VGGish with frozen embeddings and a classical classifier is the best pretrained option (0.830–0.840), far above frozen VGG16 with the same classifiers (+14.5 to +20.0 pp) and trainable in well under a second (logistic regression or SVM), though on a single partition.

Throughout all the experiments a few limitations have been identified and stated, here a summary is offered.

First of all, each test set has 200 songs (one song = 0.5 pp), and the five partitions are random splits of the same 999 songs with overlapping training sets, so the corrected resampled t-test is

conservative and has little power. A non-significant result is not evidence of equivalence. Also, Hyperparameters and configurations were chosen on partition 42 and kept on the others. The values for seed 42 are slightly optimistic, and the test songs of the other partitions partly overlap with the songs used for selection; this applies equally to all models. The best of the 12 Task 2 configurations is the maximum of 12 estimates on an 80-song validation set and is optimistic. Finally, GPU kernels are not deterministic: the same CNN configuration and seed gave test accuracies between 0.800 and 0.825, and VGG16 between 0.710 and 0.745 depending on the pipeline. Differences of 2–3 pp between single runs cannot be interpreted. Also the limitations of the dataset have to be stated. GTZAN is a small dataset with known issues documented in the literature (repeated songs and artists, some mislabelled clips), and the splits are made by song, not by artist, which can favour models that memorise; the conclusions apply to this setting and not necessarily to larger datasets.

In short, for this dataset of 999 songs, well-chosen hand-crafted descriptors with a tree ensemble are hard to beat in accuracy and cost, learned representations only match them in the best cases, and what the CNN learns is complementary enough to improve them when both are combined. For a CNN to win, we would need a much larger dataset.

A Complete Results of All Trained Models

All these results can be seen in the notebooks but they are displayed here as a summary.

A.1 Lab 1 pipelines (cross-validation)

Classifier	Selection	Features	Train acc.	CV acc. ± SD	Gap	Macro-F1
Random forest	correlation	55	0.999	0.785 ± 0.038	0.214	0.781
Random forest	none	58	0.999	0.784 ± 0.042	0.215	0.779
Voting ensemble	none	58	0.999	0.780 ± 0.056	0.219	0.778
Extra trees	none	58	0.999	0.779 ± 0.043	0.220	0.774
Extra trees	correlation	55	0.999	0.777 ± 0.047	0.222	0.774
Gradient boosting	none	58	0.999	0.772 ± 0.043	0.227	0.771
Gradient boosting	correlation	55	0.999	0.766 ± 0.035	0.233	0.766
AdaBoost	none	58	0.974	0.727 ± 0.050	0.247	0.732
Logistic regression	none	58	0.843	0.725 ± 0.063	0.118	0.721
MLP	PCA	47	0.905	0.724 ± 0.056	0.181	0.720
Logistic regression	ℓ_1	50.4	0.897	0.720 ± 0.063	0.177	0.719
Logistic regression	PCA	47	0.832	0.713 ± 0.058	0.119	0.707
MLP	none	58	0.869	0.708 ± 0.073	0.160	0.701
k-NN	LDA	9	0.999	0.706 ± 0.067	0.293	0.705
Logistic regression	ANOVA	40	0.847	0.697 ± 0.071	0.150	0.691
k-NN	ℓ_1	50.4	0.999	0.697 ± 0.044	0.302	0.699
k-NN	PCA	47	0.999	0.691 ± 0.048	0.308	0.692
k-NN	none	58	0.999	0.685 ± 0.048	0.314	0.685
Decision tree	RF importance	50	0.988	0.593 ± 0.046	0.395	0.588
Decision tree	none	58	0.860	0.590 ± 0.048	0.271	0.582
Decision tree	correlation	55	0.857	0.587 ± 0.044	0.270	0.578

Table 25: All 21 tuned Lab 1 pipelines, 10-fold CV on the 799 training songs of the main partition, sorted by CV accuracy. "Features" is the number of descriptors (or components) kept; for the ℓ_1 selection it is the mean over folds. Gap = train accuracy minus CV accuracy.

A.2 All models on the main partition

Model	Approach	Test acc.	Wilson 95 % interval	Macro-F1	Val. acc. (Task 2)
VGGish frozen + SVM	Audio	0.840	[0.783, 0.884]	0.839	0.912
VGGish frozen + RF	Audio	0.835	[0.777, 0.880]	0.833	0.900
VGGish frozen + LogReg	Audio	0.830	[0.772, 0.876]	0.829	0.900
CNN from scratch (Task 1 run)	CNN	0.825	[0.766, 0.871]	0.825	–
CNN from scratch (multi-partition run)	CNN	0.805	[0.745, 0.854]	0.800	–
Lab 1: Random Forest (+ corr.)	Descriptors	0.795	[0.734, 0.845]	0.791	–
Lab 1: Random Forest	Descriptors	0.790	[0.728, 0.841]	0.789	–
Lab 1: Extra Trees (+ corr.)	Descriptors	0.785	[0.723, 0.836]	0.780	–
EfficientNetB0: full	Vision	0.765	[0.702, 0.818]	0.768	0.775
VGG16: last 2 blocks	Vision	0.745	[0.680, 0.800]	0.740	0.812
ResNet50: full	Vision	0.745	[0.680, 0.800]	0.739	0.762
VGG16: full	Vision	0.740	[0.675, 0.796]	0.733	0.775
VGG16: last 2 blocks + SpecAugment	Vision	0.720	[0.654, 0.778]	0.715	0.762
EfficientNetB0: last block	Vision	0.715	[0.649, 0.773]	0.717	0.700
EfficientNetB0: last 2 blocks	Vision	0.715	[0.649, 0.773]	0.716	0.700
VGG16 fine-tuned (multi-partition run)	Vision	0.710	[0.644, 0.768]	0.714	–
VGG16: last block	Vision	0.705	[0.638, 0.764]	0.701	0.712
ResNet50: last 2 blocks	Vision	0.700	[0.633, 0.759]	0.704	0.750
EfficientNetB0: head only	Vision	0.695	[0.628, 0.755]	0.702	0.688
ResNet50: head only	Vision	0.690	[0.623, 0.750]	0.685	0.762
VGG16 frozen + RF	Vision	0.690	[0.623, 0.750]	0.681	0.675
ResNet50: last block	Vision	0.685	[0.618, 0.745]	0.683	0.750
VGG16 frozen + SVM	Vision	0.685	[0.618, 0.745]	0.682	0.688
VGG16: head only	Vision	0.670	[0.602, 0.731]	0.664	0.650
VGG16 frozen + LogReg (multi-partition run)	Vision	0.660	[0.592, 0.722]	0.662	–
VGG16 frozen + LogReg	Vision	0.630	[0.561, 0.694]	0.625	0.637

Table 26: All 26 models evaluated on the 200 test songs of the main partition (seed 42), sorted by test accuracy. Task 2 models are those of Section 4; "multi-partition run" marks the models retrained in the five-partition pipeline, and "Task 1 run" the CNN of Section 3.3. The validation accuracy is only available for the Task 2 search (80 validation songs).

A.3 Pairwise comparisons of the models trained on five partitions

A vs B	Acc. 42 (A / B)	Only A / only B right	McNemar p (Holm)	Mean diff. (pp)	A wins	t-test p (Holm)	Verdict
RF+corr. vs RF	0.795 / 0.790	7 / 6	1.000	+1.0	4/5	0.708	n.s.
RF+corr. vs ET+corr.	0.795 / 0.785	12 / 10	1.000	+1.6	5/5	0.151	n.s.
RF+corr. vs CNN	0.795 / 0.805	15 / 17	1.000	+5.3	4/5	0.637	n.s.
RF+corr. vs VGG16-ft	0.795 / 0.710	35 / 18	0.270	+14.8	5/5	0.065	n.s.
RF+corr. vs VGG16-frozen	0.795 / 0.660	44 / 17	0.010	+20.9	5/5	0.065	partial
RF vs ET+corr.	0.790 / 0.785	11 / 10	1.000	+0.6	3/5	0.708	n.s.
RF vs CNN	0.790 / 0.805	15 / 18	1.000	+4.3	4/5	0.708	n.s.
RF vs VGG16-ft	0.790 / 0.710	36 / 20	0.396	+13.8	5/5	0.066	n.s.
RF vs VGG16-frozen	0.790 / 0.660	44 / 18	0.017	+19.9	5/5	0.065	partial
ET+corr. vs CNN	0.785 / 0.805	18 / 22	1.000	+3.7	4/5	0.708	n.s.
ET+corr. vs VGG16-ft	0.785 / 0.710	35 / 20	0.465	+13.2	5/5	0.065	n.s.
ET+corr. vs VGG16-frozen	0.785 / 0.660	45 / 20	0.032	+19.3	5/5	0.065	partial
CNN vs VGG16-ft	0.805 / 0.710	31 / 12	0.059	+9.5	5/5	0.011	partial
CNN vs VGG16-frozen	0.805 / 0.660	41 / 12	0.001	+15.6	5/5	0.006	different
VGG16-ft vs VGG16-frozen	0.710 / 0.660	24 / 14	1.000	+6.1	5/5	0.151	n.s.

Table 27: All 15 pairs of the six models trained on five partitions (same columns as Table 15).

B Hyperparameters and Configurations

Table 28 summarises the hyperparameters of the final models and of the experiments of this report; the complete configuration search of the CNN is in Table 2 and the Lab 1 search spaces in Table 10.

Model / stage	Hyperparameters
CNN from scratch (final)	Four blocks of 3×3 convolution (same padding) + batch normalisation + ReLU + 2×2 max pooling with 32, 64, 128 and 256 filters; global average pooling; dropout 0.0; dense layer with 10 logits. Input $128 \times 192 \times 3$. AdamW (weight decay 10^{-4}), cosine learning-rate schedule from 10^{-3} over 40 epochs, 39 epochs trained, batch size 32, cross-entropy. SpecAugment-style masking: one frequency band of up to 16 rows and one time band of up to 24 columns. No validation set in the final training.
CNN search (Task 1)	16 configurations (Table 2); validation split of 160 songs; early stopping on validation accuracy (patience 10).
Pretrained vision networks (Task 2)	Input $160 \times 240 \times 3$; ImageNet weights; global average pooling + dropout 0.3 + dense softmax layer. Phase 1: head only, Adam, learning rate 10^{-3}; phase 2: last layers unfrozen (Table 6), Adam, learning rate 10^{-5}; at most 30 epochs per phase, early stopping on validation accuracy (patience 5, best weights restored); batch size 32; batch normalisation frozen; no augmentation. Multi-partition runs: VGG16 unfrozen from `block4`, validation set = 10 % of the training songs of each partition.
Classifiers on frozen embeddings (VGG16 512-d, VGGish 256-d)	Standardisation followed by logistic regression (at most 2,000 iterations in Task 2, 3,000 in the multi-partition runs), SVM (RBF kernel, probability estimates), or random forest (300 trees).
Lab 1 pipelines (selected on partition 42)	Standardisation; random forest and extra trees with 300 trees and balanced class weights; correlation filter with threshold 0.9 (random forest + correlation filter: `max_features` $= \sqrt{p}$, minimum leaf size 1; extra trees + correlation filter: the same); random forest without selection: `max_features` $= 0.2p$, minimum leaf size 1. Grids of all classifiers: Table 10.
Ensembles (Q3)	Soft voting with equal weights; "confidence-matched" variant: probabilities raised to a power (and renormalised) so that the mean top-1 probability of each model on the test songs equals 0.70 (no labels used).

Table 28: Hyperparameters of the models of this report.